

Java 企業框架實務班課程講義



一. Springboot 起手式

1.1 Springboot 源起:

1. 背景：Spring Framework 的問題

在 Spring Boot 出現之前，企業應用大多使用 **傳統的 Spring Framework** (如 Spring 3.x、4.x) 開發，但面臨以下痛點：

- **繁瑣的 XML 設定：**
 - 每個 Bean、資料庫連線、事務管理都需手動定義。
 - **組態複雜且零散：**
 - 要整合 MVC、JPA、Security、AOP，需處理大量 Java 或 XML 配置。
 - **部署繁瑣：**
 - 傳統 Web 專案需打包成 **.war** 檔，部署到外部應用伺服器 (如 Tomcat、JBoss)。
 - **缺乏快速上手的樣板：**
 - 初學者很難快速構建能運作的專案。
- **結論：**Spring 雖然彈性高、功能強，但**入門門檻高、開發效率低、配置繁雜**。

2. 解決方案：Spring Boot 誕生的目的

Spring 團隊為了解決上述問題，推出 **Spring Boot**，其目的是：

- ☐ 簡化 Spring 應用開發流程
- ☐ 去除繁瑣的配置
- ☐ 提供開箱即用的預設設定 (opinionated defaults)
- ☐ 讓應用程式能快速啟動、快速部署

3. Spring Boot 的核心設計理念

設計理念	說明
Convention over Configuration (約定大於配置)	透過慣例提供預設設定，減少人工設定負擔。
Auto-Configuration (自動配置)	根據 classpath 中的 library 自動配置對應的元件 (如 H2、MySQL、Web MVC、JPA)。
內建伺服器 (Embedded Server)	內建 Tomcat / Jetty / Undertow，不需要外部部署容器。
Spring Initializr 快速建置專案	提供官方入口 https://start.spring.io 快速生成 Maven/Gradle 專案架構。
整合 Actuator、DevTools 等工具	提供開發、監控、熱部署等輔助功能。

4. Spring Boot 的第一版

- 發布時間：2014 年
- 版本號：Spring Boot 1.0
- 所屬團隊：由 Pivotal 團隊開發 (Spring 的主要維護者)
- 目的：提升 Spring 應用的開發效率與一致性，降低進入門檻

5. Spring Boot 解決了什麼？

傳統 Spring 問題

Spring Boot 解決方案

大量 XML 設定

使用 Java Config + 自動組態

手動部署到外部伺服器

提供內建伺服器，直接用 `main()` 執行

配置多套元件需查資料手動整合

根據依賴自動載入模組配置，如 Web, JPA, Security

新手難入門、缺範例

提供 Spring Initializr + 官方起手模板

6. Spring Boot 的定位

- 並不是「取代 Spring」，而是**建立在 Spring Framework 基礎上的增強工具**。
- 提供「整合性強、配置簡單、部署容易」的開發體驗。
- 適合快速構建 RESTful API、企業系統、微服務架構。

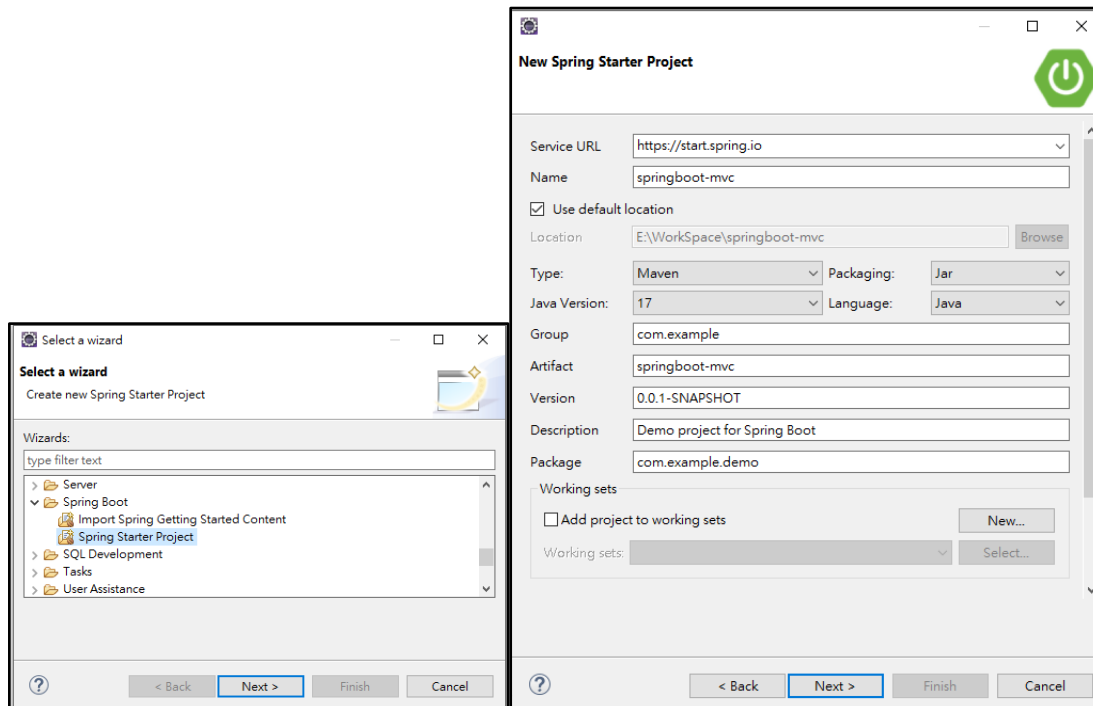
7. 後續發展

- Spring Boot 隨後成為 Spring 生態系的核心技術。
- 成為開發 **Spring Cloud 微服務架構** 的基礎模組。
- 搭配 **Spring Data**、**Spring Security**、**Spring AI** 等模組建構現代企業應用。

1.2 Springboot IDE 開發工具:

1. Eclipse EE + STS4(外掛)
2. STS4(官方下載 <https://spring.io/tools>)
3. VSCode, IntelliJ, Netbeans etc...

File > New > Other



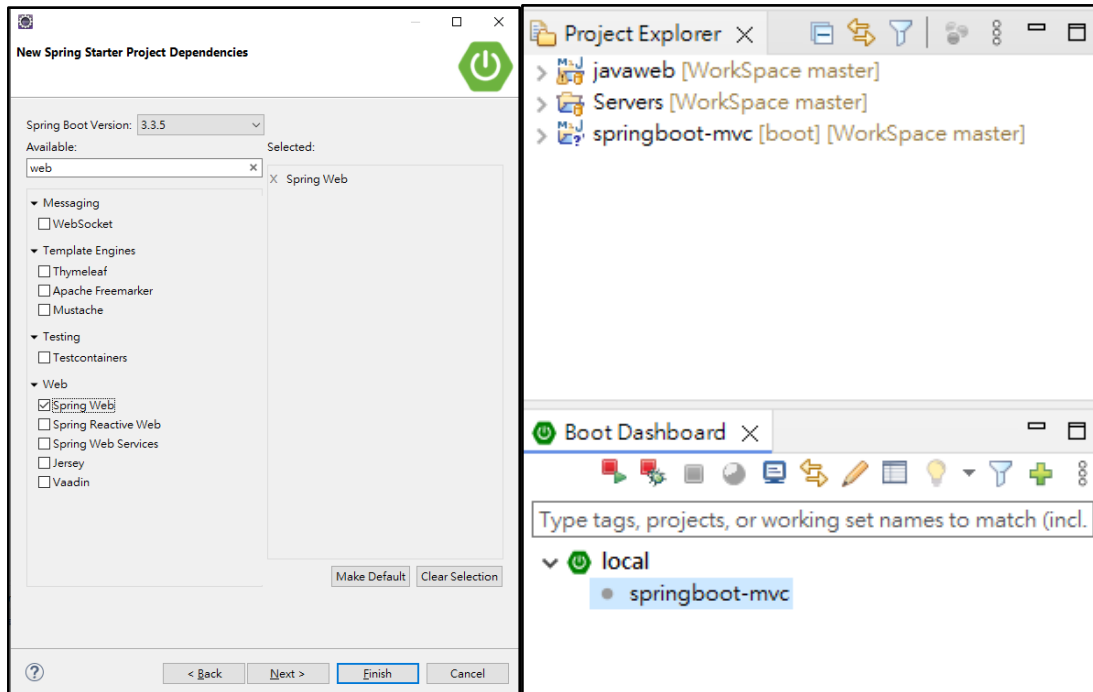
Project Name: springboot-mvc

Type : Maven

Packaging : Jar

Java Version : 17

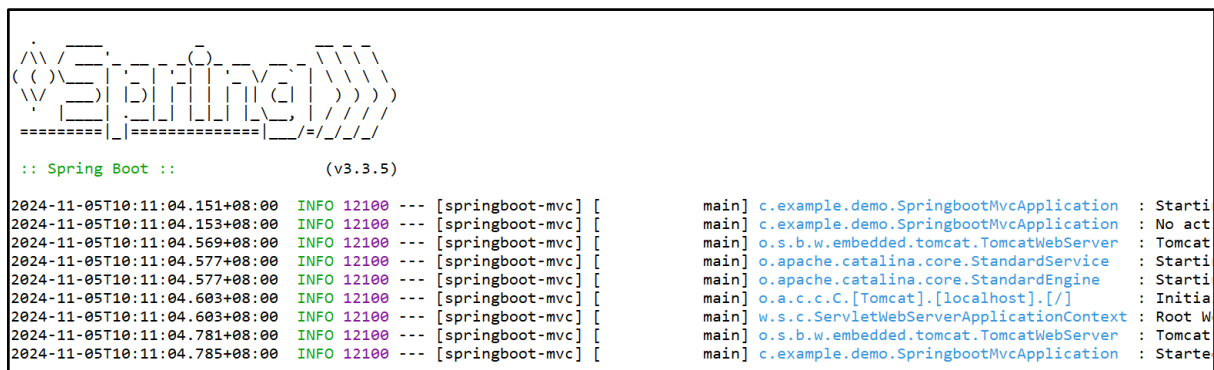
Language : Java



搜尋: web

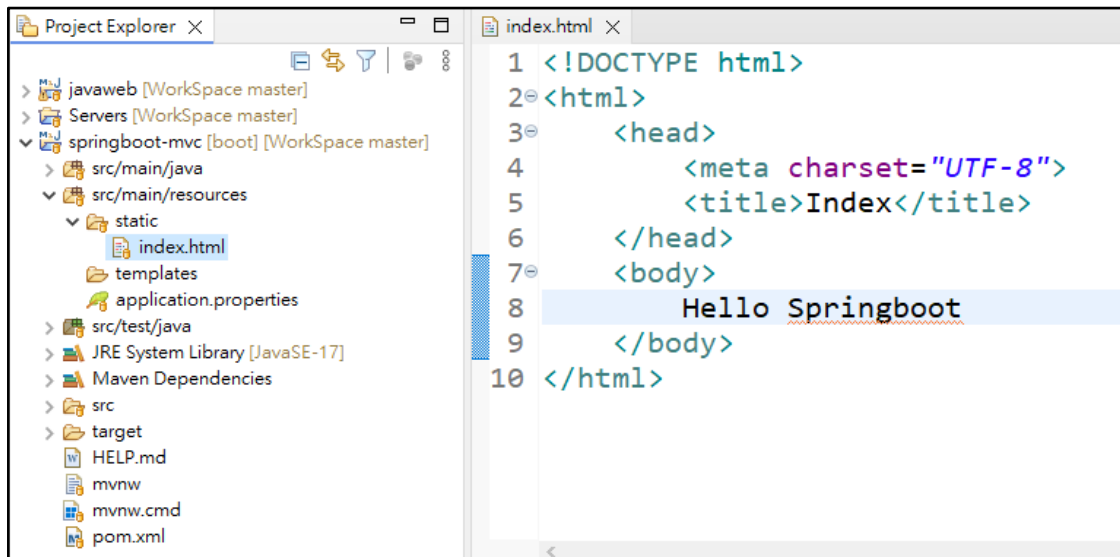
勾選: Web > Spring Web

啟動畫面



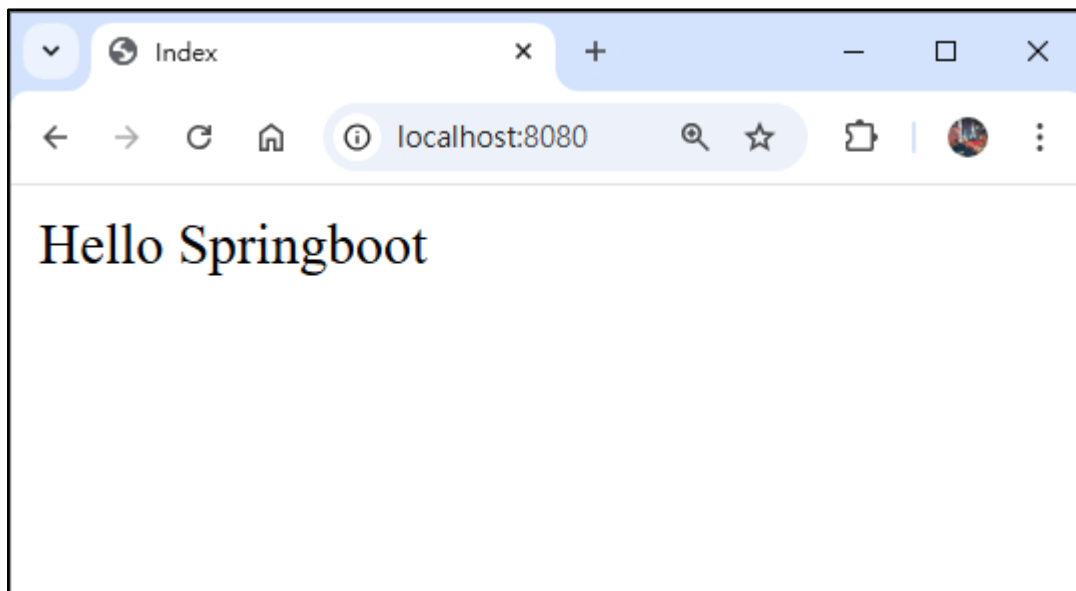
在 src/main/resources/static 下建立 index.html

static 通常放置 css, images, js, html 等靜態資料

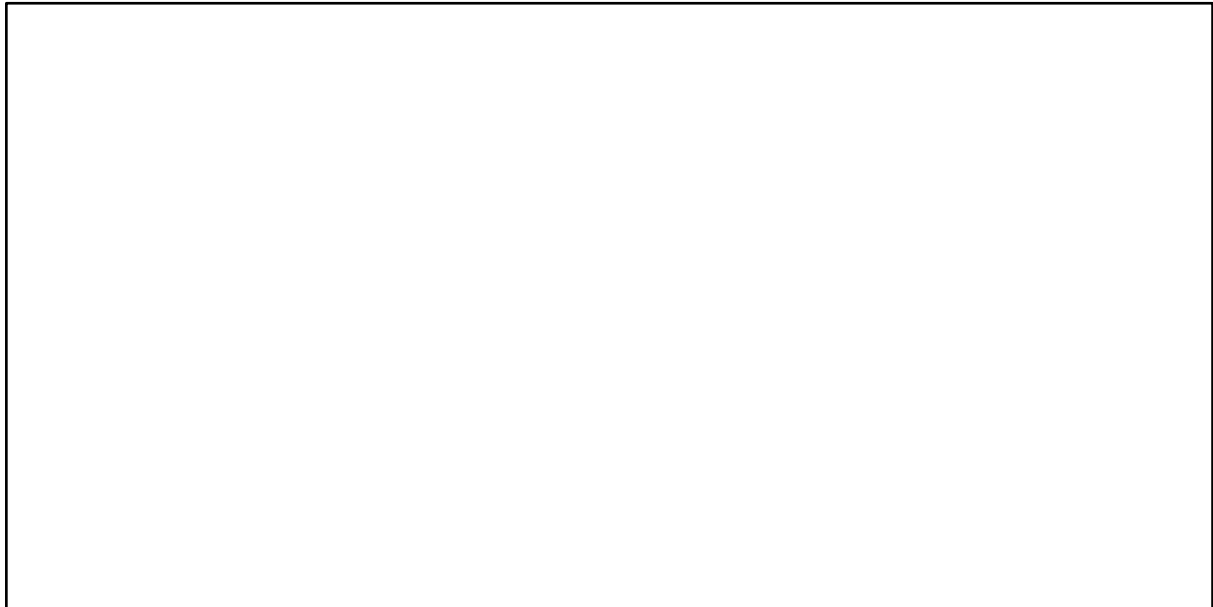


重新啟動 springboot-mvc

網址輸入 localhost:8080



1.3 我的第一個 Controller



基本配置含 JSP 與 MySQL

pom.xml 起手標配

```
<dependencies>

<!-- Model Mapper DTO 與 Entity 互轉 -->
<dependency>
  <groupId>org.modelmapper</groupId>
  <artifactId>modelmapper</artifactId>
  <version>3.2.1</version>
</dependency>

<!-- JdbcTemplate (JDBC 豪華昇級版) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
```



```
<scope>runtime</scope>
</dependency>

<!-- 配置 jsp -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<!-- 配置 Jakarta Standard Tag Library (JSTL) -->
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.0</version>
</dependency>

<!-- 配置 Tomcat Web Server -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- 配置 Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>
```

application.properties 基本配置

注意: 請先確認是否已安裝 MySQL 並建立 web 資料庫

基本配置

```
spring.application.name=springboot-mvc  
server.port=8080
```

context path (選配)

```
#server.servlet.context-path=/mvc
```

jsp 配置

```
spring.mvc.view.prefix=/WEB-INF/view/  
spring.mvc.view.suffix=.jsp
```

mysql 配置

```
spring.datasource.url=jdbc:mysql://localhost:3306/web?useSSL=false&serverTimezone=Asia/  
Taipei&useLegacyDatetimeCode=false&allowPublicKeyRetrieval=true  
spring.datasource.username=root  
spring.datasource.password=abc123  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Log 配置

根日誌層級為 INFO

```
logging.level.root=INFO
```

設定日誌保存目錄和文件名稱(會在專案目錄下自動建立一個 log 資料夾與 app.log 檔案)

```
logging.file.name=logs/app.log
```

JSP 配置



Controller + JSP 運作圖

二. Springboot 常用註解

2.1 常用的註解 (@) 簡述

以下是 Spring Boot 中常用的註解 (@)，包括其功能、目的及適用位置的簡單表格：

註解 (@)	功能	目的	使用位置
@SpringBootApplication	啟動 Spring Boot 應用程式	作為啟動類的註解，包含了 @Configuration、@EnableAutoConfiguration 和 @ComponentScan	應用程式主類
@RestController	將類註冊為 RESTful 控制器	用於處理 RESTful 請求	控制器類別
@Controller	將類註冊為 MVC 控制器	處理 Web 請求並返回頁面或視圖	控制器類別
@ControllerAdvice	將類註冊全局處理器	全局異常處理, 全局數據綁定, 全局屬性資料共享	控制器類別
@Service	將類註冊為服務層	用於業務邏輯處理	服務層類別
@Repository	將類註冊為資料訪問層	與資料庫互動，通常與 JPA 等使用	資料訪問層類別
@Component	通用註解，將類註冊為 Spring 管理的 Bean	用於標註一般組件	任何類別

@Autowired	自動注入依賴	用於注入其他 Bean	屬性、構造函數或方法
@Qualifier	指定注入的 Bean 名稱	用於避免多個同類型 Bean 的衝突	@Autowired 的屬性或參數
@Value	注入外部配置值	用於從 application.properties 加載設定	屬性或構造函數參數
@RequestMapping	指定請求 URL 路徑及方法	用於指定控制器方法的 URL 映射	控制器類別或方法
@GetMapping	處理 GET 請求	簡化的 @RequestMapping 設定 GET	控制器方法
@PostMapping	處理 POST 請求	簡化的 @RequestMapping 設定 POST	控制器方法
@PutMapping	處理 PUT 請求	簡化的 @RequestMapping 設定 PUT	控制器方法
@DeleteMapping	處理 DELETE 請求	簡化的 @RequestMapping 設定 DELETE	控制器方法
@RequestParam	從請求中獲取參數	用於提取查詢參數或表單資料	控制器方法的參數
@PathVariable	從路徑變數獲取參數	用於從 URL 路徑中提取變數	控制器方法的參數

@RequestBody	將 JSON 資料轉換為 Java 物件	用於處理 JSON 格式的請求數據	控制器方法的參數
@ResponseBody	將方法返回的資料轉換為 JSON	用於返回 JSON 格式資料，通常與 @RestController 搭配使用	控制器方法
@Entity	將類標註為 JPA 實體	用於定義資料庫中的表結構	實體類別
@Table	指定實體對應的表名	用於設置資料表名稱或 schema	實體類別
@Id	標記主鍵	用於設置實體的主鍵欄位	實體屬性
@GeneratedValue	自動生成主鍵	用於自增欄位的主鍵生成策略	主鍵屬性
@Column	指定欄位設定	用於指定資料庫欄位的名稱或約束	實體屬性
@Transactional	標記方法或類中的操作為事務性	保證資料操作的原子性	服務方法或類別
@ExceptionHandler	處理異常	用於捕捉和處理特定異常	控制器方法
@CrossOrigin	支援跨域請求	用於允許前端跨域訪問	控制器方法或類別
@Configuration	指定為設定類	用於設定 Spring Beans 或組態	配置類別

@Bean	定義一個 Bean	用於手動定義並註冊 Bean	配置類別的方法
@EnableCaching	啟用快取功能	用於啟動 Spring 的快取支持	配置類別
@Cacheable	標記快取結果	用於快取方法的返回值	服務方法
@Scheduled	設定排程任務	用於定義定期執行的任務	方法
@Async	設置非同步執行	用於標記非同步方法	方法

這些是 Spring Boot 中最常見的註解，用於不同的組態和邏輯層。

2.2 @Controller 與 @RestController 使用時機(一般來說)

@Controller 會將資料透過 JSP 或 Thymeleaf 來渲染 - 走 SSR 架構

@RestController 會將資料透過 JSON 來傳遞 - 走 CSR 架構

JSON 再透過前端 JS 技術來渲染到 HTML 網頁

@RestController = @Controller + @ResponseBody

混和模式 @Controller + @ResponseBody (在必要時)

混和模式 = SSR + 部分的 Ajax 技術 ← 早期的開發方式

2.3 UTF-8 編碼與 js 設定

@GetMapping(value = "/age", produces = "text/plain;charset=utf-8")

若回傳的是 json 則 produces 要改為

produces = "application/json;charset=utf-8"

2.4 超簡單 Log 配置起手

利用 Springboot 所提供的 org.slf4j 來配置

使用 log 就可取代傳統的 System.out.println()

application.properties 加入

```
# 根日誌層級為 INFO
logging.level.root=INFO

# 設定日誌保存目錄和文件名稱
logging.file.name=logs/app.log
```

使用方式範例(以 ApiController.java 為例):

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
@RequestMapping("/api")
public class ApiController {

    private static final Logger logger = LoggerFactory.getLogger(ApiController.class);

    @GetMapping(value = {"/welcome", "/home"})
    public String welcome() {
        logger.info("這是一條日誌訊息");
        return "Welcome";
    }
}
```


三. SpringMVC

3.1 什麼是 Spring MVC ?

- Spring MVC 是 Spring Framework 中提供的一個 **Web 應用程式框架**。
- 它實作了 **MVC 架構模式** (Model-View-Controller)，用來處理 Web 應用的請求路由、業務邏輯與畫面顯示。
- Spring MVC 是基於 Servlet API 的封裝與擴展。

3.2 MVC 三層架構說明

1. Model (模型層)

- 負責資料的結構與業務邏輯處理。
- 常對應 Java 類別、DTO、Entity 或與資料庫連接的物件。
- 不參與顯示與控制流程，但提供資料給 View。

2. View (視圖層)

- 負責畫面呈現。
- 常見技術有 Thymeleaf、JSP、Freemarker 等。
- 接收 Controller 提供的資料，渲染給使用者。

3. Controller (控制器)

- 負責處理 HTTP 請求。
- 接收來自瀏覽器或客戶端的請求，呼叫 Model 層處理後，將結果回傳給 View。
- 在 Spring MVC 中使用 `@Controller` 或 `@RestController` 標註。

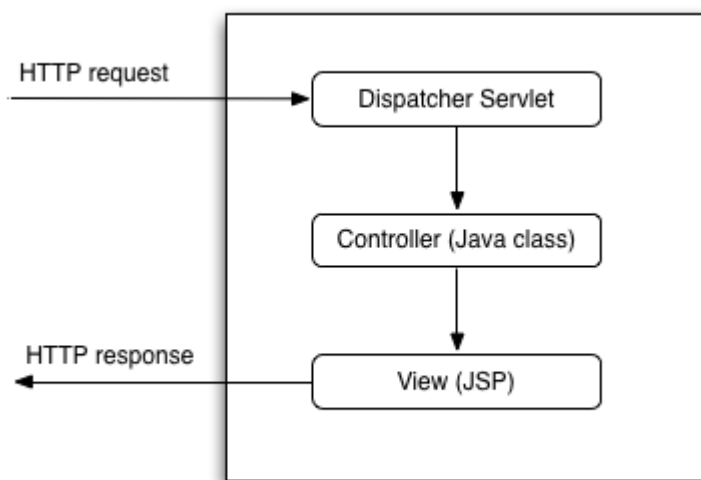
3.3 請求處理流程 (Request Flow)

1. 用戶端發送請求 (如：HTTP GET /products)
2. **DispatcherServlet (中央調度器)** 會攔截所有請求
3. DispatcherServlet 根據 URL 尋找對應的 Controller 方法
4. 執行 Controller 方法，調用必要的 Service/Model
5. 返回 View 名稱或 JSON 資料

6. 若為 View，交由 ViewResolver 將資料渲染成 HTML
7. 回應結果傳給用戶端

3.4 DispatcherServlet 是什麼？

- 是 Spring MVC 的 核心 Servlet。
- 所有的 HTTP 請求都會進入這個類別進行統一處理。
- 在 Spring Boot 中自動配置為 / 路徑的預設 servlet。



3.5 SpringMVC Layer 說明

Layer	Description	Common Annotations
Model	負責管理應用程式中的數據和業務邏輯，並將數據傳遞到視圖。	@Model, @ModelAttribute, @Entity, @Table, @Id, @Column
View	負責顯示 Model 中的數據，通常使用模板引擎（如 Thymeleaf、JSP）。	@RequestParam, @ResponseBody, @JsonView
Controller	處理 HTTP 請求和響應，並將 Model 和 View 聯繫起來。	@Controller, @RestController, @RequestMapping, @GetMapping, @PostMapping, @PathVariable,

		@RequestBody
Service	定義業務邏輯，處理應用的核心功能。	@Service
Repository (DAO)	負責數據訪問和數據庫操作 (CRUD 操作) 。	@Repository, @Transactional

3.6 Spring MVC 特點總結

- 基於 Servlet 與 DispatcherServlet 架構。
- 使用註解驅動 (Annotation-based) 簡化配置。
- 支援 RESTful API 設計 (可搭配 @RestController) 。
- 可整合 Thymeleaf、JSP 等模板引擎。
- 分離資料處理 (Model)、邏輯控制 (Controller) 與畫面呈現 (View)，有利於維護與測試。

3.7 Spring MVC 適合場景

- 傳統 Web 應用 (表單送出、頁面跳轉)
- SPA 頁面中後端 API 資料支援 (與 Vue、React、Angular 搭配)
- RESTful API 架構
- 結合資料庫操作與商業邏輯的企業應用程式

四. CSR-Book 圖書查詢系統(InMemory + Restful)

4.1 URI 設計

HTTP 方法	路徑	說明
GET	/book	查詢所有書籍
GET	/book/1	查詢指定書籍
POST	/book	新增書籍
PUT	/book/1	完整修改書籍
PATCH	/book/1	部分修改書籍(name+price)
PATCH	/book/price/1	部分修改書籍(price)
PATCH	/book/name/1	部分修改書籍(name)
DELETE	/book/1	刪除書籍

4.2 MVC 模式配置

model > Book.java

controller > BookController.java

service > BookService.java(介面), BookServiceImpl.java

repository >

BookRepository.java (介面)

BookRepositoryImpl.java(實現類 - InMemory)

BookRepositoryJdbcImpl.java(實現類 - JDBC MySQL)

BookRepository.java

```
package com.example.demo.repository;
import java.util.List;
import java.util.Optional;
import com.example.demo.model.Book;
public interface BookRepository {
    List<Book> findAllBooks();
    Optional<Book> getBookById(Integer id);
    boolean addBook(Book book);
    boolean updateBook(Integer id, Book book);
    boolean deleteBook(Integer id);
}
```

BookService.java

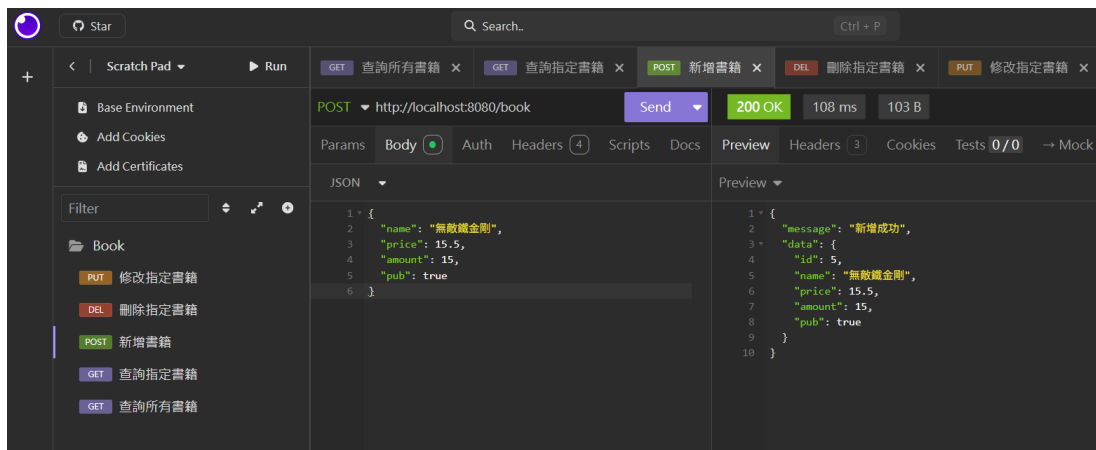
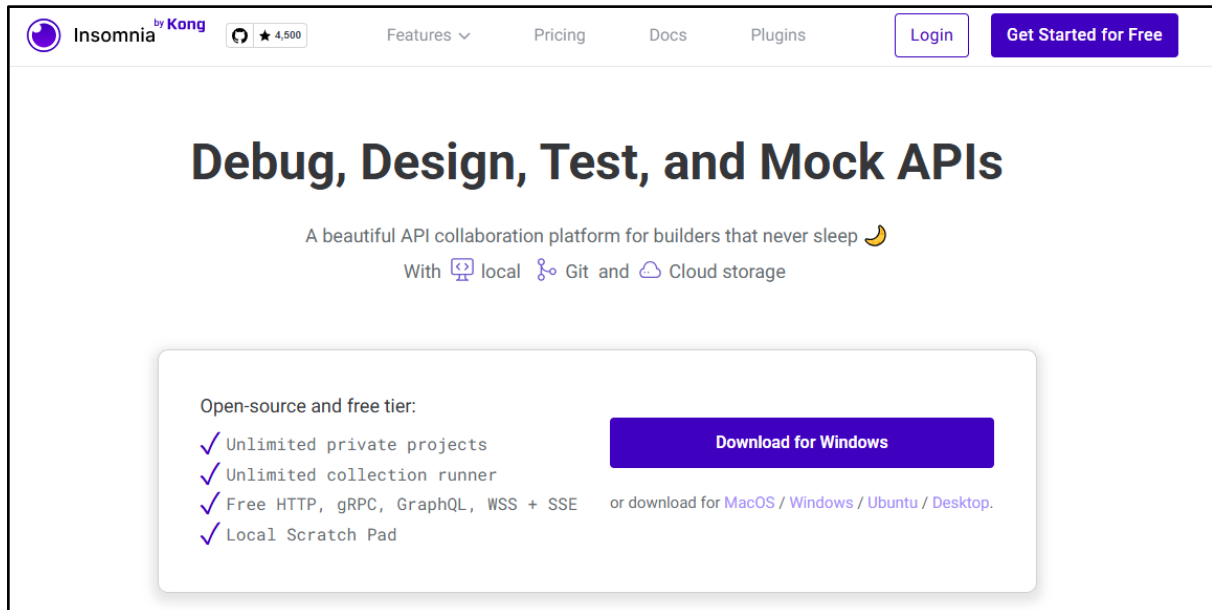
```
package com.example.demo.service;
import java.util.List;
import com.example.demo.exception.BookException;
import com.example.demo.model.Book;
public interface BookService {
    List<Book> findAllBooks();
    Book getBookById(Integer id) throws BookException;
    void addBook(Book book) throws BookException;

    void updateBook(Integer id, Book book) throws BookException;
    void updateBookName(Integer id, String name) throws BookException;
    void updateBookPrice(Integer id, Double price) throws BookException;
    void updateBookNameAndPrice(Integer id, String name, Double price) throws
BookException;

    void deleteBook(Integer id) throws BookException;
}
```

4.3 測試 Restful 的工具

安裝 <https://insomnia.rest/>



4.4 使用 React 前端框架

使用 React 前端框架 (函式庫) 去實現：Create (新增)、Read (讀取)、Update (更新)、Delete (刪除) 的 資料管理功能，並搭配後端 API (如 Spring Boot REST API) 完成資料交換。

□ 書籍管理系統 (使用 fetch)

[書名 :[]]

[價格 :[]]

[數量 :[]]

[出刊 :[]]

[[新增書籍] [取消] (如果在編輯狀態才出現)]

書籍列表

ID	書名	價格	數量	出刊	操作
----	----	----	----	----	----

1	Java	380	10	是	[編輯] [刪除]
---	------	-----	----	---	-----------

2	Python	420	5	否	[編輯] [刪除]
---	--------	-----	---	---	-----------

五. CSR-Book 圖書查詢系統(MySQL + Restful)

5.1 MySQL 資料表建置與配置

```
use web;

CREATE TABLE if not exists book (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  price DECIMAL(5, 1) NOT NULL,
  amount INT NOT NULL,
  pub BOOLEAN NOT NULL
);

insert into book(name, price, amount, pub) values("小叮嚀", 12.5, 20, false);
insert into book(name, price, amount, pub) values("老夫子-水虎傳", 10.5, 30, false);
insert into book(name, price, amount, pub) values("頑皮豹", 8.5, 40, true);
insert into book(name, price, amount, pub) values("小甜甜", 14.5, 50, true);
```

為什麼要用 **DECIMAL(5,1)** 而不是 **DOUBLE** ?

▣ 重點：金額數值請避免使用 FLOAT/DOUBLE !

資料型別	精準度	適用場景
DOUBLE / FLOAT	近似值 (非精確)	科學計算、圖形處理等
DECIMAL(p,s)	精確值	金融、價格、交易相關

DECIMAL(5,1) 的意思

- 5 是總位數 (包含整數與小數)
- 1 是小數位數 (例如 999.9)
- 可表示範圍 : -999.9 ~ 999.9

application.properties 配置

```
# mysql 配置
spring.datasource.url=jdbc:mysql://localhost:3306/web?useSSL=false&serverTimezone=Asia/Taipei&useLegacyDatetimeCode=false&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=abc123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

pom.xml

```
<!-- JdbcTemplate (JDBC 豪華昇級版) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>

<!-- MySQL Driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

5.2 JdbcTemplate 簡介

JdbcTemplate 是 Spring Framework 提供的一個資料存取工具，屬於 Spring JDBC 模組。它簡化了使用 JDBC 操作資料庫時的繁瑣程式碼，例如建立連線、資源釋放、錯誤處理等。

優點

1. 自動處理資源釋放 (Connection 、 Statement 、 ResultSet)
2. 減少樣板式程式碼
3. 支援參數化 SQL (防止 SQL Injection)

5.3 BookRepositoryJdbcImpl 實現

```
package com.example.demo.repository;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Primary;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import com.example.demo.model.Book;

@Repository
//@Primary
public class BookRepositoryJdbcImpl implements BookRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate; // 自動綁定 spring 內建的 JdbcTemplate 物件

    @Override
    public List<Book> findAllBooks() {
        String sql = "select id, name, price, amount, pub from book";
        // BeanPropertyRowMapper(Book.class) 自動將每一筆紀錄注入到 Book 物件中
        return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(Book.class));
    }

    @Override
    public Optional<Book> getBookById(Integer id) {
        String sql = "select id, name, price, amount, pub from book where id=?";
        try {
            // 查單筆
            Book book = jdbcTemplate.queryForObject(sql,
                                                    new BeanPropertyRowMapper<>(Book.class), id);
            return Optional.of(book);
        } catch (EmptyResultDataAccessException e) {
            // 沒查到資料會拋出例外
            return Optional.empty();
        }
    }
}
```

```
}

@Override
public boolean addBook(Book book) {
    String sql = "insert into book(name, price, amount, pub) values(?, ?, ?, ?)";
    int rows = jdbcTemplate.update(sql,
        book.getName(), book.getPrice(), book.getAmount(), book.getPub());
    return rows > 0;
}

@Override
public boolean updateBook(Integer id, Book book) {
    String sql = "update book set name = ?, price = ?, amount = ?, pub = ? where id = ?";
    int rows = jdbcTemplate.update(sql,
        book.getName(), book.getPrice(), book.getAmount(), book.getPub(), id);
    return rows > 0;
}

@Override
public boolean deleteBook(Integer id) {
    String sql = "delete from book where id = ?";
    int rows = jdbcTemplate.update(sql, id);
    return rows > 0;
    //return jdbcTemplate.update("delete from book where id = ?", id) > 0;
}
}
```

Service 切換不同 Repository 的實現

```
@Autowired
@Qualifier("bookRepositoryImpl") // Bean 名稱(類名字首小寫)
private BookRepository bookRepository;
```

```
@Autowired
@Qualifier("bookRepositoryJdbcImpl") // Bean 名稱(類名字首小寫)
private BookRepository bookRepository;
```

六. SSR-Book 圖書查詢系統(MySQL + Restful)

6.1 URI 設計

HTTP 方法	路徑	說明
GET	/ssr/book	查詢所有書籍頁面
GET	/ssr/book/{id}	查詢指定書籍頁面
POST	/ssr/book/add	新增書籍
GET	/ssr/book/edit/{id}	顯示編輯頁面
POST	/ssr/book/edit/{id}	修改書籍
GET	/ssr/book/delete/{id}	刪除書籍

6.2 JSP 相關配置

pom.xml 依賴

```
<!-- 配置 jsp -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<!-- 配置 Jakarta Standard Tag Library (JSTL) -->
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.0</version>
</dependency>
```

application.properties 配置

```
# jsp 配置  
spring.mvc.view.prefix=/WEB-INF/view/  
spring.mvc.view.suffix=.jsp
```

View 配置

src/main/webapp/

└─ WEB-INF/

└─ view/

└─ include/

| └─ menu.jsp ← 目錄

└─ book-list.jsp ← 顯示所有書籍(內含新增表單)

└─ book-edit.jsp ← 修改書籍

└─ error.jsp ← 錯誤頁面

6.3 SSRBookController 實現

定義 URI

```
@Controller
@RequestMapping( "/ssr/book" )
public class SSRBookController {

}
```

6.4 什麼是 HiddenHttpMethodFilter ?

HiddenHttpMethodFilter 是 Spring Web 提供的一個 Servlet Filter，它的作用是讓 HTML 表單 (form) 支援非 GET / POST 的 HTTP 方法，例如：

- PUT
- DELETE
- PATCH

這是因為：

- HTML 表單只支援 GET 和 POST 方法，並不支援 PUT、DELETE 等。

□ 原理：使用 _method 隱藏欄位達成方法轉換

當你使用 Spring Boot 並註冊 HiddenHttpMethodFilter，可以透過以下 HTML 表單方式，模擬 PUT 或 DELETE：

```
<form action="/book/1" method="post">
  <input type="hidden" name="_method" value="PUT"/>
  <!-- 其他欄位 -->
  <button type="submit">更新</button>
</form>
```

這段表單看起來是 POST，但實際上透過 HiddenHttpMethodFilter，會被轉換成 PUT 方法！

設定 application.properties

```
# 啟用 hiddenmethod filter
spring.mvc.hiddenmethod.filter.enabled=true
```

□ 實際運作流程

1. 用戶透過表單送出 **POST** 請求
2. 表單中包含:
`<input type="hidden" name="_method" value="PUT"/>`
3. `HiddenHttpMethodFilter` 攔截到這個請求，發現 `_method=PUT`
4. 將原始請求 `method` 改為 **PUT**
5. Spring Controller 對應的 `@PutMapping` 被觸發

修改 URI 設計

HTTP 方法	路徑	說明
GET	/ssr/book	查詢所有書籍頁面
GET	/ssr/book/{id}	查詢指定書籍頁面
POST	/ssr/book/add	新增書籍
GET	/ssr/book/edit/{id}	顯示編輯頁面
POST 改 PUT	/ssr/book/edit/{id}	修改書籍
GET 改 DELETE	/ssr/book/delete/{id}	刪除書籍

七. JPA 基礎

理解和掌握 Java Persistence API (JPA) 並學會如何在 Spring Boot 專案中使用它來進行資料庫操作。如何建立資料庫實體類、使用 JPA 的 Repository 進行基本的 CRUD 操作、設計資料表之間的關聯等。

- 熟悉 Java 語言
- 基本了解 SQL 和資料庫操作
- 熟悉 Spring Boot 基本架構

1. JPA 概述

- 什麼是 JPA ?
- JPA 與 ORM (Object-Relational Mapping) 的關係
- JPA 與 Hibernate 的區別
- JPA 在 Spring Boot 中的應用

2. 安裝與設定 Spring Boot + JPA

- 如何在 Spring Boot 中整合 JPA
- 配置資料庫連線 (使用 MySQL 為範例)
- 使用 `spring-boot-starter-data-jpa` 啟用 JPA 功能

3. 建立資料庫實體類 (Entity Class)

- JPA 實體類的基本結構
- 設定主鍵、欄位和資料表映射

4. JPA Repository

- 使用 `JpaRepository` 進行基本的 CRUD 操作
- 自定義查詢方法 (例如 : `findBy`、`deleteBy` 等)

5. JPA 關聯

- 一對一關聯 (One-to-One)
- 一對多關聯 (One-to-Many)
- 多對多關聯 (Many-to-Many)

6. 事務管理與資料庫交易

- 如何在 JPA 中管理事務
- 事務的開啟、提交與回滾

7. 進階使用

- 查詢語言 JPQL (Java Persistence Query Language)
 - 本地查詢 (Native SQL)
 - 分頁與排序
 - 屬性映射 (`@Column` 等)
-

7.1 JPA 概述

1. 什麼是 JPA ?

JPA (Java Persistence API) 是 Java EE (現在的 Jakarta EE) 規範，旨在使 Java 應用程式可以以物件導向的方式管理關聯資料庫資料。JPA 定義了對象與資料庫表之間的映射，讓開發者能夠使用 Java 物件來表示資料庫資料，而不需要直接寫 SQL 查詢語句。

2. JPA 與 ORM 的關係

JPA 本身並不是一個實作，它是一個規範。常見的 JPA 實作是 **Hibernate**，這是目前使用最廣泛的 ORM 框架。ORM (Object-Relational Mapping) 是一種技術，使得開發者可以將資料庫中的資料與 Java 物件進行映射。

3. JPA 與 Hibernate 的區別

- JPA：是 Java 官方的規範（接口）。
- Hibernate：是一個 JPA 的實作，它提供了 JPA 規範中定義的功能和更多的擴展功能。

4. JPA 在 Spring Boot 中的應用

Spring Boot 提供了 `spring-boot-starter-data-jpa`，它幫助開發者輕鬆整合 JPA 和 Hibernate。Spring Data JPA 會自動生成常用的 CRUD 操作，讓開發者無需手動寫繁瑣的 SQL。

7.2 安裝與設定 Spring Boot + JPA

1. 建立 Spring Boot 專案

- 使用 Spring Initializr (<https://start.spring.io/>) 生成專案，選擇以下依賴：
 - Spring Web
 - Spring Data JPA
 - MySQL Driver

2. 配置 `application.properties`

MySQL JPA 資料庫連線設定

`spring.jpa.hibernate.ddl-auto=update` # 自動更新資料庫結構

`spring.jpa.database-platform=org.hibernate.dialect.MySQL5Dialect`

3. 資料庫準備

確保你已經在 MySQL 中建立了資料庫。例如，建立 web 資料庫：

```
CREATE DATABASE web;
```

7.3 建立資料庫實體類 (Entity Class)

1. 定義 `Book` 實體類

```
import javax.persistence.*;
```

```
@Entity
```

```
public class Book {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY) // 主鍵自增
```

```
    private Integer id;
```

```
private String name;

private Double price;

private Integer amount;

private Boolean pub;

// Getter 和 Setter 省略
}
```

- `@Entity` 表示這個類別是 JPA 實體，會對應到資料庫表格。
- `@Id` 表示主鍵。
- `@GeneratedValue` 設定主鍵自動生成策略。

2. 資料表結構

JPA 會根據 `Book` 類的屬性自動生成資料表。如果你在資料庫中沒有這個表，它會被自動創建。

```
CREATE TABLE book (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  price DOUBLE,
  amount INT,
  pub BOOLEAN
);
```

7.4 JPA Repository

1. 使用 JpaRepository

Spring Data JPA 提供了 `JpaRepository` 來簡化資料庫操作。這個介面包含了許多 CRUD 方法，例如：`findAll()`, `findById()`, `save()`, `deleteById()`。

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository<Book, Integer> {
    // 自動提供基本的 CRUD 操作
}
```

2. 基本 CRUD 操作範例

```
@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    // 查詢所有書籍
    public List<Book> findAllBooks() {
        return bookRepository.findAll();
    }

    // 新增書籍
    public Book addBook(Book book) {
        return bookRepository.save(book);
    }

    // 刪除書籍
```

```
public void deleteBook(Integer id) {  
    bookRepository.deleteById(id);  
}  
}
```

7.5 JPA 方法命名關鍵詞

Spring Data JPA 能識別並自動產生 SQL 的關鍵詞 (Keywords) ，可以在方法名中使用：

1. 查詢條件比較類 (條件運算)

關鍵字	說明	範例方法
<code>findBy, readBy, getBy</code>	開頭 (查詢)	<code>findByName(String name)</code>
<code>Is, Equals</code>	等於	<code>findByAgeEquals(int age)</code>
<code>Between</code>	介於	<code>findByAgeBetween(int min, int max)</code>
<code>LessThan / Before</code>	小於	<code>findByAgeLessThan(int age)</code>
<code>GreaterThan / After</code>	大於	<code>findBySalaryGreaterThan(Double salary)</code>
<code>Like</code>	模糊比對 (%name%)	<code>findByNameLike(String name)</code>
<code>StartingWith / StartsWith</code>	前綴比對	<code>findByNameStartingWith(String prefix)</code>
<code>EndingWith / EndsWith</code>	後綴比對	<code>findByNameEndingWith(String suffix)</code>
<code>Containing / Contains</code>	包含字串	<code>findByNameContaining(String keyword)</code>
<code>In</code>	在集合中	<code>findByIdIn(List<Integer> ids)</code>
<code>Not</code>	非 (否定)	<code>findByNameNot(String name)</code>
<code>IsNull / Null</code>	為 null	<code>findByDescriptionIsNull()</code>
<code>IsNotNull / NotNull</code>	不為 null	<code>findByNameIsNotNull()</code>

2. 排序與限制條件

關鍵字	說明	範例方法
OrderBy	排序	findByAgeGreaterThanOrderByAgeDesc()
Top, First	只取前幾筆	findTop1ByOrderBySalaryDesc()
Distinct	去除重複	findDistinctByDepartment(String dept)

3. 邏輯運算子 (AND / OR)

關鍵字	說明	範例方法
And	且	findByNameAndAge(String name, int age)
Or	或	findByNameOrSalary(String name, double salary)

八. Java Proxy 代理

Java 的 Proxy (代理) 設計模式主要分為兩種實現方式：靜態 Proxy 與動態 Proxy 。兩者都用於在不改變原有業務邏輯的前提下，為對象提供額外的功能（如日誌、權限控制等），但實作方式與適用場景有所不同。

8.1 靜態 Proxy (Static Proxy)

定義與原理

靜態 Proxy 是指由開發者手動撰寫代理類，這個代理類與目標類實現相同的介面，並在代理類的方法中包裝對目標類的調用，同時可以在調用前後插入額外邏輯（如日誌、權限檢查等）

優缺點

- 優點：結構簡單，易於理解，適合代理類型較少時使用。
- 缺點：每新增一個被代理類都要手動撰寫對應的代理類，當介面方法增多或被代理類型增多時，會產生大量重複代碼，維護困難

8.2 動態 Proxy (Dynamic Proxy , JDK 1.3+ 支援)

定義與原理

動態 Proxy 是指在程式運行時，透過 Java 反射與 API 動態生成代理類，不需手動撰寫代理類。JDK 1.3 開始支援，核心類為 `java.lang.reflect.Proxy` 與 `InvocationHandler`。

主要步驟

1. 定義介面（與靜態 Proxy 相同）
2. 實現業務邏輯類
3. 實現 `InvocationHandler`

在 `invoke` 方法中定義代理邏輯（如日誌、權限等），並透過反射調用目標方法。

4. 使用 `Proxy.newProxyInstance()` 生成代理對象

JDK 動態 Proxy 的底層原理

- 透過 `Proxy.newProxyInstance()` 方法動態生成一個實現指定介面的代理類，並將 `InvocationHandler` 綁定到代理實例上。
- 代理類會將所有介面方法的調用轉發到 `InvocationHandler.invoke()` 方法，由開發者決定是否、何時以及如何調用原始方法。
- 動態生成的代理類會繼承 `Proxy` 類並實現所有指定的介面，所有方法最終都會調用 `InvocationHandler.invoke()`。

優缺點

- 優點：無需為每個被代理類手動撰寫代理類，減少重複代碼，易於維護，適合大量代理場景。
- 缺點：只能代理介面（JDK 動態 Proxy），無法代理類本身（如需代理類，需使用 CGLIB 等技術）；動態生成類有一定性能開銷

8.3 靜態 Proxy 與動態 Proxy 比較表

特性	靜態 Proxy	動態 Proxy (JDK)
代理類產生	手動撰寫	程式運行時自動生成
代理對象	需實現相同介面	需實現相同介面
維護成本	高（多個類需多份代理）	低（通用 <code>InvocationHandler</code> ）
擴展性	差	好

性能	較高（無反射開銷）	較低（需反射調用）
適用場景	代理類型少，邏輯簡單	代理類型多，需統一增強

九. Spring AOP

Spring AOP (Aspect-Oriented Programming , 面向切面程式設計) 是一種用於將橫切關注點 (如日誌、權限、交易等) 從業務邏輯中分離出來的技術。這讓開發者能在不改動原有業務代碼的情況下，動態地為方法添加額外行為。

pom.xml 加入:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

9.1 核心概念

- 切面 (Aspect) :
橫切關注點的模組化表現，通常是一個類，裡面包含多個通知 (Advice) 。
- 通知 (Advice) :
定義在切點 (Pointcut) 上執行的動作，如前置 (@Before) 、後置 (@After) 、環繞 (@Around) 等。
- 切點 (Pointcut) :
定義哪些方法會被攔截，通常用表達式指定。
- 連接點 (Join Point) :
程式執行過程中可插入切面的點，Spring AOP 主要支援方法層級的連接點。
- 目標對象 (Target Object) :
被增強的業務對象。
- 代理 (Proxy) :
AOP 框架為目標對象生成的代理對象，用來包裝目標對象並攔截方法調用。

9.2 Spring AOP 的實現原理

Spring AOP 的底層實現基於代理模式，主要採用兩種動態代理技術：

1. JDK 動態代理

- 適用於目標類實現了介面的情況。
- 代理類會實現與目標類相同的介面，並在方法調用前後插入切面邏輯。
- 優點：不需第三方庫，效率較高。
- 限制：只能代理介面中定義的方法，若類沒有介面則無法使用。

2. CGLIB 動態代理

- 適用於目標類沒有實現介面的情況。
- 透過生成目標類的子類，並重寫其方法來插入切面邏輯（基於 ASM 操作位元組碼）。
- 優點：可代理沒有介面的類。
- 限制：不能代理 final 類和 final 方法，效率略低於 JDK 動態代理。

Spring AOP 會自動判斷使用哪種代理方式：如果 bean 實現了介面，預設使用 JDK 動態代理，否則使用 CGLIB 代理。

代理流程簡述

1. 定義切面：使用 `@Aspect` 標註類，並用 `@Pointcut`、`@Before`、`@After`、`@Around` 等註解標記方法。
 2. Spring 容器啟動時，掃描所有切面類與目標類，解析切點表達式與通知方法。
 3. Bean 實例化後，Spring 根據切點規則為符合條件的 bean 生成代理對象（JDK 或 CGLIB）。
 4. 方法調用時，實際執行的是代理對象的方法，代理邏輯會在適當時機呼叫原始方法並執行通知。
-

與 AspectJ 的關係

Spring AOP 只支援運行時動態代理（即方法級別的攔截），而 AspectJ 則支援編譯時、載入時與運行時的靜態與動態織入，功能更強大。Spring AOP 受 AspectJ 設計影響，並可與 AspectJ 註解整合使用。

Spring AOP Pointcut EL 語法說明

Spring AOP 的切點（Pointcut）表達式語法採用 AspectJ 標準，主要用於精確指定哪些方法（Join Point）會被攔截並套用橫切邏輯（Advice）。這些表達式以**切點指示符（Pointcut Designator, PCD）**開頭，最常用的是 `execution`，此外還有 `within`、`this`、`target`、`args`、`@annotation` 等

常用 Pointcut Designator 與語法

Designator	用途與語法範例
execution	匹配方法執行： <code>execution(public String com.example.FooService.findById(Long))</code> 可用萬用字元： <code>execution(* com.example.FooService.*(..))</code> （匹配所有方法）
within	匹配指定類型內所有方法： <code>within(com.example.service.*)</code>

this	匹配代理對象型別： this(com.example.service.FooService)
target	匹配目標對象型別： target(com.example.service.FooService)
args	匹配參數型別： args(java.io.Serializable) 匹配參數為 Serializable 的方法
@annotation	匹配方法上標註特定註解： <code>@annotation(org.springframework.transaction.annotation.Transactional)</code>
@within	匹配類上標註特定註解： @within(org.springframework.stereotype.Service)
@target	匹配目標對象類型標註特定註解：

	@target(org.springframework.stereotype.Repository)
@args	匹配參數類型標註特定註解

execution 語法詳解

execution(modifiers-pattern? ret-type-pattern declaring-type-pattern?name-pattern(param-pattern) throws-pattern?)

- **modifiers-pattern**：如 public、private，可用 * 匹配全部。
- **ret-type-pattern**：返回型別，如 String、*。
- **declaring-type-pattern**：類的全名或萬用字元。
- **name-pattern**：方法名，可用 * 匹配全部或部分。
- **param-pattern**：參數型別，如 () (無參)、(String)、(..) (任意數量參數)、(*,String) (第一個任意，第二個 String)。
- **throws-pattern**：可選，匹配 throws 子句。

範例：

- 匹配所有 public 方法：`execution(public **(..))`
- 匹配 com.example.service 下所有類的所有方法：`execution(* com.example.service.*(..))`
- 匹配方法名以 find 開頭、參數為 Long：`execution(* *.find*(Long))`
- 匹配第一個參數為 Long 的方法：`execution(* *.find*(Long,..))`

表達式組合

可用 && (AND)、|| (OR)、! (NOT) 組合多個切點

範例：

```
@Pointcut("@target(org.springframework.stereotype.Repository)")
```



```
public void repositoryMethods() {}

@Pointcut("execution(* *..create*(Long,...))")
public void firstLongParamMethods() {}

@Pointcut("repositoryMethods() && firstLongParamMethods()")
public void entityCreationMethods() {}
```

9.3 Spring AOP 適用場景

- 日誌記錄
- 權限驗證
- 交易管理
- 性能監控
- 緩存處理

Spring AOP 透過代理模式 (JDK 動態代理與 CGLIB) 在運行時動態生成代理對象，將橫切邏輯與業務邏輯分離，實現程式碼的高度可維護性與重用性，是現代 Java 應用中處理橫切關注點的主流方案。

十. JPA Room(SSR+CSR) Lab 練習

10.1 Springboot Web 專案目錄配置

以 Room 房間管理為範例

建立一個 (SSR-JSP 版) WebApp 可以對 Room 進行 CRUD

Room 欄位 (room_id 房號, room_name 房名, room_size 使用人數)

```
src/main/java
├── com
│   └── example
│       └── demo
│           ├── controller          # 控制器層，處理 HTTP 請求和響應
│           │   ├── RoomController.java      # 控制器類，負責處理 Room 相關的 CRUD 請求
│           │   └── RoomRestController.java  # 控制器類，負責處理 Room 相關的 REST CRUD 請求
│           │
│           ├── service            # 服務層，包含業務邏輯
│           │   ├── RoomService.java        # 服務介面，定義 Room 的業務操作
│           │   └── impl            # 服務實現層（可選），具體實現服務介面
│           │       └── RoomServiceImpl.java # 服務介面的實現類，包含業務邏輯
│           │
│           ├── repository         # 資料庫訪問層，與資料庫交互
│           │   ├── RoomRepository.java     # 資料庫介面，擴展 JpaRepository 提供 CRUD
│           │   ├── RoomRepositoryJdbc.java # JdbcTemplate 資料庫介面
│           │   └── impl            # 
│           │       └── RoomRepositoryJdbcImpl.java # JdbcTemplate 資料庫介面 增強版實現
│           │
│           ├── model              # 放置數據模型，包括 entity 和 dto
│           │   ├── entity         # 實體類，映射到資料庫表
│           │   │   └── Room.java   # Room 實體類，映射到資料庫的 Room 表
│           │   │
│           │   └── dto            # 數據傳輸對象（DTO），用於數據傳輸
│           │       └── RoomDTO.java # Room 的 DTO 類，用於控制器與服務之間的數據傳輸
│           │
│           ├── config             # 配置類，用於各種應用程序設置
│           │   ├── WebConfig.java    # Web 配置類，如 CORS 設置等
│           │   ├── ModelMapperConfig.java # 配置 ModelMapper，便於 DTO 和實體之間的轉換
│           │   └── SecurityConfig.java # 安全配置類（如 Spring Security 設置）
│           │
│           └── exception          # 自定義例外處理類
```

```

|   |— RoomException.java      # 自定義例外，用於 Room 根例外
|   |— RoomAlreadyExistsException.java # 自定義例外，用於 Room 已存在時拋出
|   |— RoomNotFoundException.java  # 自定義例外，用於用於 Room 未找到時拋出
|   |— GlobalExceptionHandler.java # 全局例外處理器，統一處理其他例外
|
|— validation      # 放置自定義驗證器
|   |— RoomNameValidator.java # 自定義驗證器類，用於驗證房間名稱的格式
|   |— ValidRoomName.java    # 自定義驗證註解，標記需驗證的字段
|
|— util            # 工具類，包含通用的輔助方法
|   |— DateUtil.java        # 日期工具類，處理日期相關的操作
|
|— mapper          # 放置 DTO 和 Entity 轉換器 ( 可選 )
|   |— RoomMapper.java      # Room 的轉換器類，處理 Room 和 RoomDTO 的互轉

```

```

src/main/resources      # 存放應用的靜態資源、配置文件等
|— static               # 靜態資源目錄，如 CSS、JS、圖片等
|   |— css              # 存放 CSS 文件
|   |— js               # 存放 JavaScript 文件
|— templates            # 模板文件目錄 ( 如使用 Thymeleaf )
|   |— room.html        # Room 相關的 HTML 模板
|— messages.properties  # 默認語言 ( 如英文 ) 的消息文件
|— application.properties # 全局應用配置文件
|— sql.properties       # 全局 SQL 語法彙集區
|— logback.xml          # 日誌配置文件，用於設置日誌格式和級別

```

```

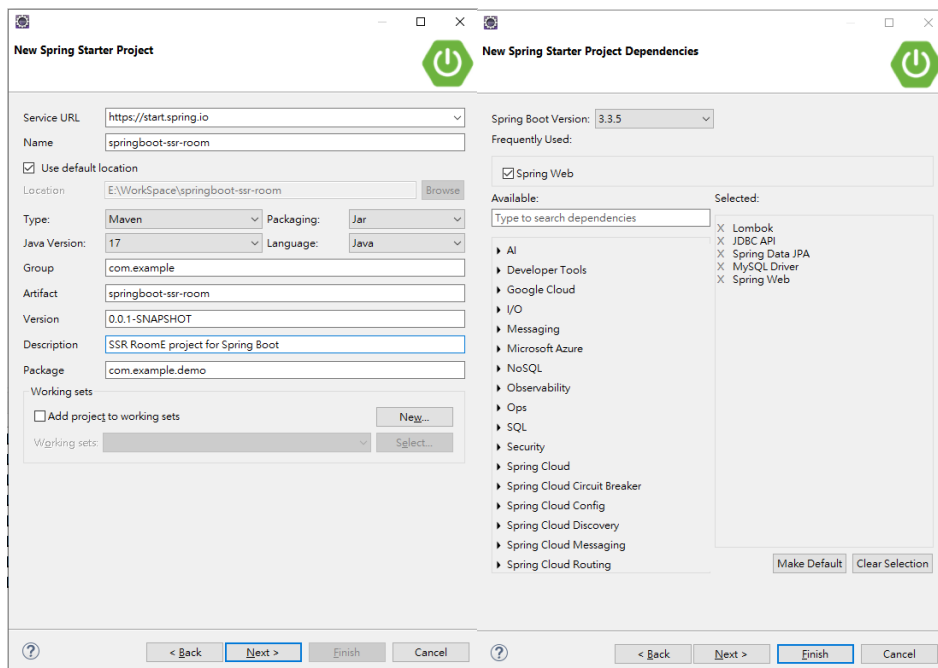
src/main/webapp         # 根目錄，放置靜態資源和 Web 頁面，主要針對傳統 web 應用
|— WEB-INF              # Web 應用程序的非公開目錄
|   |— view             # 放置 JSP 視圖文件的目錄(jspf)
|       |— room_form.jspf # JSPF 文件，用於顯示 room 新增表單的視圖內容
|       |— room_list.jspf # JSPF 文件，用於顯示 room 列表的視圖內容
|       |— room_update.jspf # JSPF 文件，用於顯示 room 修改表單的視圖內容
|       |— room.jsp      # 具體的 JSP 文件，用於顯示 room 相關的視圖內容

```

.jsp vs .jspf 比較表

項目	.jsp	.jspf (JSP Fragment)
用途	網頁頁面本體，可單獨執行	可被包含的 JSP 區塊，不能獨立執行
可否執行	☐ 可以直接被請求並執行	☐ 不能直接請求執行，只能被包含使用
包含方式	可包含其他 JSP 或 JSPF	通常被其他 JSP 用 <code><%@ include %></code> 包含
副檔名	.jsp	.jspf (表示 fragment)
典型用途	實際的頁面邏輯，例如 <code>/login.jsp</code>	頁首、頁尾、導覽列、表格區塊模板等
範例	<code>/index.jsp</code>	<code>/includes/header.jspf</code>

10.2 建立 Springboot Starter 專案



手動加入 modelmapper 與 jsp/jstl 依賴

```
<dependencies>

<!-- Model Mapper DTO 與 Entity 互轉 -->
<dependency>
  <groupId>org.modelmapper</groupId>
  <artifactId>modelmapper</artifactId>
  <version>3.2.1</version>
</dependency>

<!-- 配置 jsp -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<!-- 配置 Jakarta Standard Tag Library (JSTL) -->
<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>3.0.0</version>
</dependency>

... 略 (以下不要刪除)

</dependencies>
```

application.properties

```
# 基本配置
spring.application.name=springboot-ssr-room
server.port=8081

# jsp 配置
spring.mvc.view.prefix=/WEB-INF/view/
spring.mvc.view.suffix=.jsp
```

```
# mysql 配置
spring.datasource.url=jdbc:mysql://localhost:3306/web?useSSL=false&serverTimezone=Asia/Taipei&useLegacyDatetimeCode=false
spring.datasource.username=root
spring.datasource.password=abc123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA 配置
# 自動更新表結構，可根據需要設置為 create, update, validate, none
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

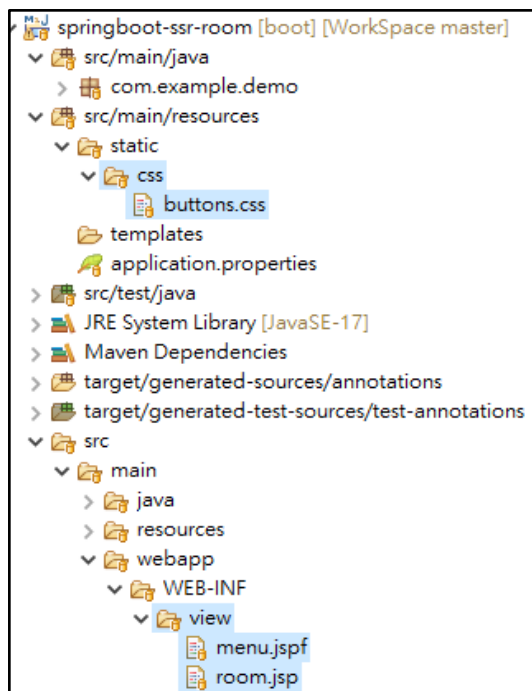
# Log 配置
# 根日誌層級為 INFO
logging.level.root=INFO
# 設定日誌保存目錄和文件名稱(會在專案目錄下自動建立一個 log 資料夾與 app.log 檔案)
logging.file.name=logs/app.log
```

jsp 與 css 起手配置

src/main/resources/static/css/buttons.css

src/main/webapp/WEB-INF/view/menu.jspf

src/main/webapp/WEB-INF/view/room.jsp



10.3 @Entity 取代手動撰寫 SQL 語句

model # 放置數據模型，包括 entity 和 dto
└─ entity # 實體類，映射到資料庫表
 └─ Room.java # Room 實體類，映射到資料庫的 Room 表

```
@Entity // 實體類與資料表對應(會自動建立資料表)
@Table(name = "room") // 若資料表名與實體類一致可以不用設定此行
public class Room {

    @Id
    //room_id 自動生成, 從 1 開始每次自動 +1 過號不補
    //@GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "room_id")
    private Integer roomId;

    @Column(name = "room_name", nullable = false, unique = true)
    private String roomName;

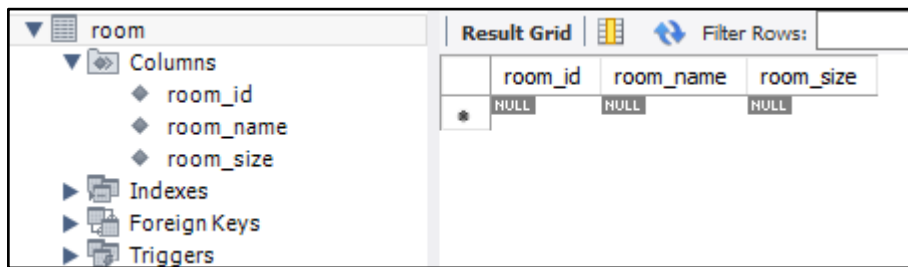
    @Column(name = "room_size", columnDefinition = "integer default 0")
    private Integer roomSize;

}
```

上述程式碼會自動產生以下新增 SQL 語句並交由 MySQL 執行

```
create table room(
    room_id integer primary key,
    room_name varchar(50) not null unique,
    room_size integer default 0
);
```


執行後於 MySQL 中可以看到



The screenshot shows the MySQL Workbench interface. On the left, the 'room' table is selected, and its structure is displayed under 'Columns', 'Indexes', 'Foreign Keys', and 'Triggers'. The 'Columns' section shows three columns: 'room_id', 'room_name', and 'room_size'. On the right, the 'Result Grid' shows the data for the 'room' table. It has three columns: 'room_id', 'room_name', and 'room_size'. The first row shows three NULL values, indicating that the table is currently empty.

	room_id	room_name	room_size
*	NULL	NULL	NULL

建立 RoomDTO

model # 放置數據模型，包括 entity 和 dto

└─ dto # 數據傳輸對象 (DTO)，用於數據傳輸

 └─ RoomDTO.java # Room 的 DTO 類，用於控制器與服務之間的數據傳輸

建立 Entity 與 DTO 之間的轉換

利用 modelmapper 第三方套件

設定 ModelMapperConfig

config # 配置類，用於各種應用程序設置

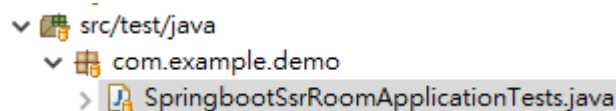
└─ ModelMapperConfig.java # 配置 ModelMapper，便於 DTO 和實體之間的轉換

轉寫 Room 的轉換器

mapper # 放置 DTO 和 Entity 轉換器（可選）

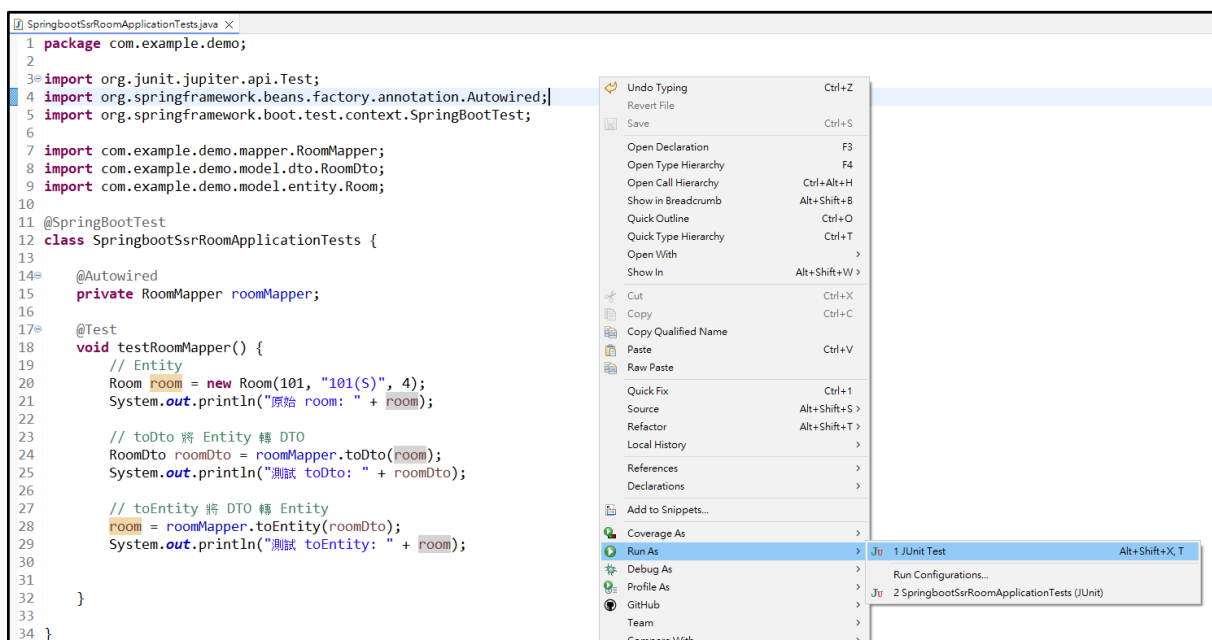
└─ RoomMapper.java # Room 的轉換器類，處理 Room 和 RoomDTO 的互轉

建立測試 RoomMapper



執行測試

程式碼空白處按下滑鼠右鍵 > Run As > JUnit Test



測試結果

原始 room: Room(roomId=101, roomName=101(S), roomSize=4)

測試 toDto: RoomDto(roomId=101, roomName=101(S), roomSize=4)

測試 toEntity: Room(roomId=101, roomName=101(S), roomSize=4)

10.4 實踐 Repository

```

repository          # 資料庫訪問層，與數據庫交互
├── RoomRepository.java # 資料庫介面，擴展 JpaRepository 提供 CRUD
├── RoomRepositoryJdbc.java # JdbcTemplate 資料庫介面
└── impl
    └── RoomRepositoryJdbcImpl.java # JdbcTemplate 資料庫介面 增強版實現
    
```

Spring Data JPA

是基於 JPA 的一個 Spring Data 模組，提供了一系列簡單的方法來處理常見的資料庫操作。使用 Spring Data JPA，開發人員基本上無需撰寫 SQL 查詢，透過方法命名即可自動生成 SQL 查詢。

在 JPA 中，Java 類別可以對應到資料庫中的表，這些類別被稱為 **Entity**。要定義一個實體類別，需要使用 **@Entity** 註解並指定主鍵欄位，請參考：

```

model              # 放置數據模型，包括 entity 和 dto
├── entity          # 實體類，映射到資料庫表
└── Room.java      # Room 實體類，映射到資料庫的 Room 表
    
```

建立 RoomRepositoryJdbc 介面

根據存取 Room 的需求(最基本 CRUD)

透過 RoomRepositoryJdbcImpl.java 實現 RoomRepositoryJdbc 介面

@Repository 表示是一個資料存取物件

```

// 實現 RoomRepositoryJdbc 介面
@Repository // 交由 Springboot 來管理此物件
public class RoomRepositoryJdbcImpl implements RoomRepositoryJdbc {
    // 預設自動實現 CRUD
    // 自定義查詢
    // 1. 查詢 roomSize 大於指定值的房間(自動生成 SQL)
    List<Room> findByRoomSizeGreaterThan(Integer size);
}
    
```

```
// 2. 查詢 roomSize 大於指定值的房間(JPQL 以 entity 來操作)
@Query("select r from Room r where r.roomSize > :size")
List<Room> findRoomsBySizeGreaterThan1(@Param("size") Integer size);

// 3. 查詢 roomSize 大於指定值的房間(SQL 以 table 來操作)
@Query(value = "select room_id, room_name, room_size from room where room_size
> :size",
      nativeQuery = true)
List<Room> findRoomsBySizeGreaterThan2(@Param("size") Integer size);

}
```

JPQL (Java Persistence Query Language)

JPQL 是 JPA 提供的一種查詢語言，用於查詢 Entity 的物件屬性。它與 SQL 相似，但操作的是 Entity 而非資料庫中的表格。

查詢命名規則

Spring Data JPA 支援以命名方法的方式撰寫查詢，透過解析方法名稱自動生成 SQL 查詢。

以下是一些命名規則：

1. 查詢關鍵字：`findBy`、`countBy`、`existsBy` 等。
2. 條件運算：`And`、`Or`、`Between`、`LessThan`、`GreaterThan` 等。
3. 屬性名稱：方法名稱中的屬性必須與 Entity 屬性名稱一致，Spring Data 會自動對應到資料庫中的欄位。

10.5 實踐自訂例外

exception	# 自定義例外處理類
└─ RoomException.java	# 自定義例外，用於 Room 根例外
└─ RoomAlreadyExistsException.java	# 自定義例外，用於 Room 已存在時拋出
└─ RoomNotFoundException.java	# 自定義例外，用於 Room 未找到時拋出
└─ GlobalExceptionHandler.java	# 全局例外處理器，統一處理其他例外

10.6 實踐 Service

service # 服務層，包含業務邏輯
└─ RoomService.java # 服務介面，定義 Room 的業務操作
 └─ impl # 服務實現層（可選），具體實現服務介面
 └─ RoomServiceImpl.java # 服務介面的實現類，包含業務邏輯

RoomService 介面規格

```
public interface RoomService {  
  
    public List<RoomDto> getAllRooms(); // 查詢所有房間  
    public RoomDto getRoomById(Integer roomId); // 查詢單筆房間  
  
    public void addRoom(RoomDto roomDto); // 新增房間  
    public void addRoom(Integer roomId, String roomName,  
                        Integer roomSize); // 新增房間  
  
    public void updateRoom(Integer roomId, RoomDto roomDto); // 修改房間  
    public void updateRoom(Integer roomId, String roomName,  
                        Integer roomSize); // 修改房間  
  
    public void deleteRoom(Integer roomId); // 刪除房間  
  
}
```

10.7 實踐 Controller

controller # 控制器層，處理 HTTP 請求和響應

└─ RoomController.java # 控制器類，負責處理 Room 相關的 CRUD 請求

Method	URI	功能
GET	/room/{roomId}	查詢指定會議室(單筆)
GET	/rooms	查詢所有會議室(多筆)
POST	/room	新增會議室
POST	/room/update/{roomId}	完整修改會議室(同時修改 roomId 與 roomSize)
GET	/room/delete/{roomId}	刪除會議室

10.8 實踐 JSP

```
src/main/webapp      # 根目錄，放置靜態資源和 Web 頁面，主要針對傳統 web
應用
└─ WEB-INF          # Web 應用程序的非公開目錄
    └─ view          # 放置 JSP 視圖文件的目錄
        └─ room_form.jspf  # JSPF 文件，用於顯示 room 新增表單的視圖內容
        └─ room_list.jspf  # JSPF 文件，用於顯示 room 列表的視圖內容
        └─ room_update.jspf # JSPF 文件，用於顯示 room 修改表單的視圖內容
        └─ room.jsp      # 具體的 JSP 文件，用於顯示 room 相關的視圖內容
```

JSP 表單引入 Spring Form 表單標籤

```
<!-- Spring Form 表單標籤 -->
<%@ taglib prefix="sp" uri="http://www.springframework.org/tags/form"%>
```

room_form.jspf

path=xxx 是必要屬性配置會自動生成 id=xxx 與 name=xxx (HTML 表單標籤屬性)

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
pageEncoding="UTF-8"%>
<!-- Spring Form 表單標籤 -->
<%@ taglib prefix="sp" uri="http://www.springframework.org/tags/form"%>

<sp:form class="pure-form" method="post" modelAttribute="roomDto" action="/room"
>
    <fieldset>
        <legend>Room 表單</legend>
        Room 房號: <sp:input type="number" path="roomId"/> <p />
        Room 名稱: <sp:input type="text" path="roomName"/> <p />
        Room 人數: <sp:input type="number" path="roomSize"/> <p />
        <button type="submit" class="pure-button pure-button-primary">新增</button>
    </fieldset>
</sp:form>
```

pring Form 表單標籤庫

<%@ taglib prefix="sp" uri="http://www.springframework.org/tags/form" %>

是用來引入 **Spring Form 表單標籤庫**，也就是 Spring Framework 提供的 JSP 標籤，用來簡化與表單資料綁定的工作。

Spring Form 標籤對應說明

標籤	用途
<sp:form>	表單本體，綁定 modelAttribute
<sp:input>	一般輸入欄位 (text)
<sp:select>	下拉選單
<sp:checkbox>	勾選框
<sp:radiobutton>	單選按鈕
<sp:errors>	顯示欄位錯誤訊息

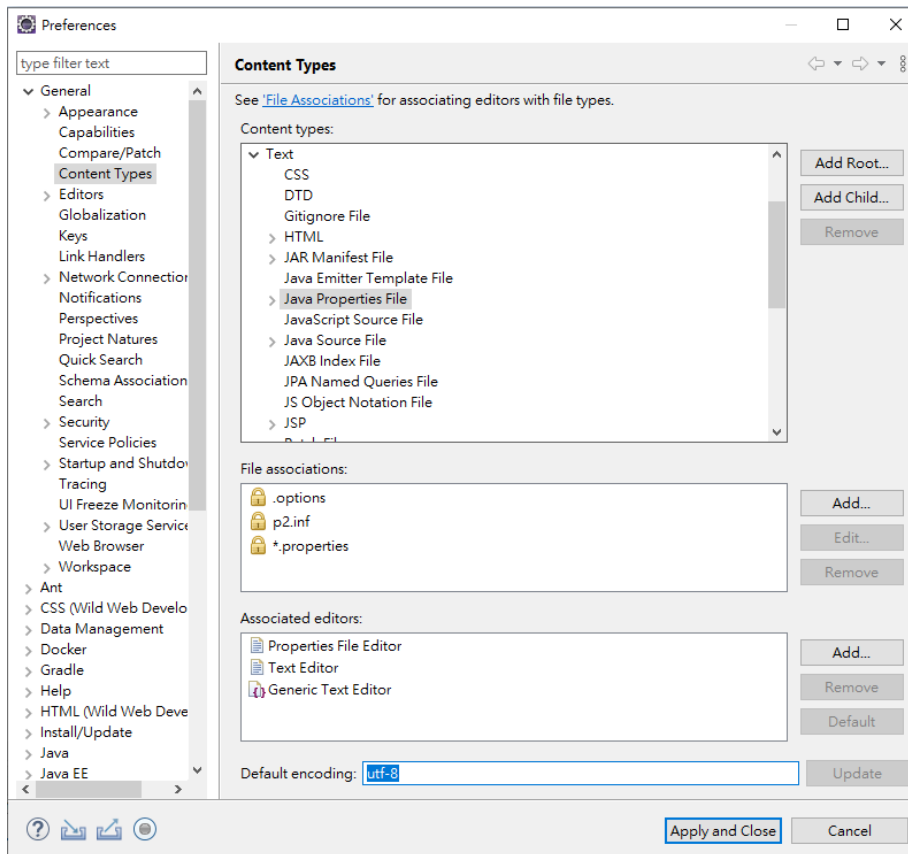
10.9 實踐表單驗證

透過 JSR 303 進行表單驗證

pom.xml 依賴

```
<!-- Spring Boot Validation Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

設定 properties 檔案編碼 utf-8



將所有錯誤訊息統一集中管理

src/main/resources/messages.properties

```
roomDto.roomId.notNull=房號不可為空
roomDto.roomName.notNull=房名不可為空
roomDto.roomName.size=房名最少要有 2 個字
roomDto.roomSize.notNull=人數不可為空
roomDto.roomSize.range=房間人數必須介於 {min} ~ {max} 人之間
```

application.properties 訊息配置設定

```
# message 配置
spring.messages.basename=messages
spring.messages.encoding=UTF-8
```

RoomDto.java (標註 @ 驗證)

```
public class RoomDto {  
  
    @NotNull(message = "{roomDto.roomId.notNull}")  
    private Integer roomId;  
  
    @NotNull(message = "{roomDto.roomName.notNull}")  
    @Size(min = 2, message = "{roomDto.roomName.size}")  
    private String roomName;  
  
    @NotNull(message = "{roomDto.roomSize.notNull}")  
    @Range(min = 1, max = 500, message = "{roomDto.roomSize.range}")  
    private Integer roomSize;  
  
}
```

RoomController.java

```
@PostMapping  
// @Valid 進行驗證  
// BindingResult 驗證結果  
public String addRoom(@Valid @ModelAttribute RoomDto roomDto,  
    BindingResult bindingResult, Model model) {  
    if(bindingResult.hasErrors()) { // 若有錯誤發生  
        model.addAttribute("roomDtos", roomService.getAllRooms());  
        return "room/room"; // 會自動將錯誤訊息傳給 jsp  
    }  
    roomService.addRoom(roomDto);  
    return "redirect:/rooms"; // 重導到 /rooms 頁面  
}
```

room_form.jspf (加入 `<sp:errors>` 標籤)

```
<!-- Spring Form 表單標籤 -->
<%@ taglib prefix="sp" uri="http://www.springframework.org/tags/form"%>
<sp:form class="pure-form" method="post"
    modelAttribute="roomDto" action="/room">
    <fieldset>
        <legend>Room 表單</legend>
        Room 房號: <sp:input type="number" path="roomId"/>
        <sp:errors path="roomId" style="color: red"/>
    <p />
        Room 名稱: <sp:input type="text" path="roomName"/>
        <sp:errors path="roomName" style="color: red"/>
    <p />
        Room 人數: <sp:input type="number" path="roomSize"/>
        <sp:errors path="roomSize" style="color: red"/>
    <p />
        <button type="submit"
            class="pure-button pure-button-primary">新增</button>
    </fieldset>
</sp:form>
```

10.10 CSR - 實作 RestBookController 與 React

後台 - RestAPI 如下:

請求方法	URL 路徑	功能說明	請求參數	回應內容
GET	/rest/room	取得所有房間列表	無	成功時返回所有房間的列表 payload 及成功訊息。
GET	/rest/room/{roomId}	取得指定房間資料	roomId (路徑參數, 房間 ID)	成功時返回指定房間資料及 payload 成功訊息。
POST	/rest/room	新增房間	請求體包含 roomDto (JSON 格式)	成功時返回成功訊息, 並包含 payload。
PUT	/rest/room/{roomId}	更新指定房間資料	roomId (路徑參數), 請求體包含 roomDto (JSON 格式)	成功時返回成功訊息, 並包含 payload。
DELETE	/rest/room/{roomId}	刪除指定房間	roomId (路徑參數, 房間 ID)	成功時返回成功訊息, 不包含 payload。

注意: CrossOrigin 要設定

```
@CrossOrigin(origins = {"http://localhost:5173", "http://localhost:8002"}, allowCredentials = "true")
public class RoomRestController {
```

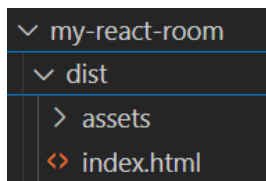
前台- 框架 React

10.11 前台部署

將寫好的 React 打包並部署到 WebServer 中 (以 Springboot 為例)

指令 `npm run build`

執行後會自動產生 `dist` 目錄



建立一個 Springboot 專案:

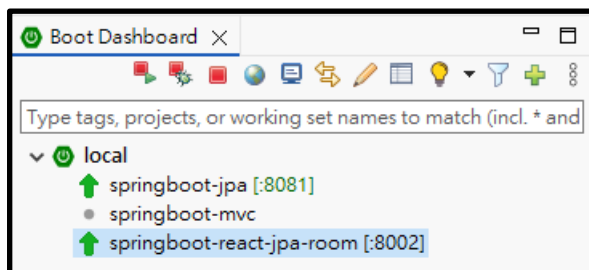
並複製 react 專案下的 `dist` 資料夾內容到 springboot 專案下的 `static` 目錄中:

`application.properties`

```
spring.application.name=springboot-react-jpa-room  
server.port=8002
```

後端: 8081

前端: 8002



十一. 登入驗證

11.1 登入驗證架構:

11.2 專案規劃與建立

```
src/main/java
├── com
│   ├── example
│   │   ├── demo
│   │   │   ├── controller
│   │   │   │   ├── LoginController.java # 控制器類: 登入驗證
│   │   │   │   |
│   │   │   │   ├── filter
│   │   │   │   │   ├── LoginFilter.java # 過濾器類: 判斷是否有登入
│   │   │   │   │   |
│   │   │   │   ├── mapper
│   │   │   │   │   ├── UserMapper.java # Entity 與 DTO 轉換
│   │   │   │   │   |
│   │   │   │   ├── exception
│   │   │   │   │   ├── CertException.java # 憑證例外
│   │   │   │   │   ├── UserNotFoundException.java # 無此使用者例外
│   │   │   │   │   └── PasswordInvalidService.java # 密碼錯誤例外
│   │   │   │   |
│   │   │   │   ├── service
│   │   │   │   │   ├── UserService.java # 服務介面, 定義 User 的業務操作
│   │   │   │   │   ├── CertService.java # 服務介面, 定義 Cert 的業務操作
│   │   │   │   │   └── impl
│   │   │   │   │       ├── UserServiceImpl.java # 實現 User 的業務操作
│   │   │   │   │       └── CertServiceImpl.java # 實現 Cert 的業務操作
│   │   │   │   |
│   │   │   |
```



```
|— repository      # 資料庫訪問層，與數據庫交互
|   |— UserRepository.java # 資料庫介面，擴展 JpaRepository 提供 CRUD
|
|— util            # 公用功能
|   |— Hash.java # Hash 雜湊程式
|
|— model
|   |— entity
|       |— User.java # User 實體類並對應資料表
|       |
|       |— dto
|           |— userDto.java # 對應於 User Entity: 前端渲染使用
|           |— userCert.java # 使用者憑證: 驗證是否有登入時使用
|
src/main/webapp    # 根目錄，放置靜態資源和 Web 頁面，主要針對傳統 web 應用
|   |— WEB-INF    # Web 應用程序的非公開目錄
|       |— view    # 放置 JSP 視圖文件的目錄(jspf)
|           |— login.jsp # 具體的 JSP 文件，用於顯示 login 相關的視圖內容
```

撰寫順序

exception.CertException.java

exception.PasswordInvalidException.java

exception.UserNotFoundException.java

model.entity.User.java

model.dto.UserDto.java

model.dto.UserCert.java

mapper.UserMapper.java

repository.UserRepository.java

service.UserService.java

service.CertService.java

service.impl.UserServiceImpl.java

service.impl.CertServiceImpl.java

controller.LoginController.java

controller.UserController.java ← 自行設計

filter.LoginFilter

注意: 若是使用 JavaWeb 版的 @WebFilter 則須加註 @ServletComponentScan 的配置

```
@SpringBootApplication
// 啟用 WebFilter 掃描, 因為 Springboot 預設會忽略 JavaWeb 基本的 @
@ServletComponentScan
public class SpringbootJpaApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringbootJpaApplication.class, args);
    }
}
```

11.3 動動腦 CSR 加入登入驗證機制

```
src/main/java
├── com
│   └── example
│       └── demo
│           ├── controller
│           │   └── LoginRestController.java # 控制器類: 登入驗證(Rest 版)
│           └── filter
│               └── LoginRestFilter.java # 過濾器類: 判斷是否有登入(Rest 版)
```

要求

未登入時僅可以看見所有房間

登入後才能進行房間管理(CRUD)

Rest login API

名稱	方法	路徑	請求參數	回應 json
登入	POST	/rest/login	username=XXX & password=XXX	{ "status": 200, "message": "登入成功", "data": null }
登出	GET	/rest/logout		{ "status": 200, "message": "登出成功", "data": null }
檢查登入	GET	/rest/check-login		{ "status": 200, "message": "檢查登入", "data": true }

十二. JPA 關聯關係

關聯類型	註解	是否需要 <code>@JoinColumn</code>	是否需要中介表	備註
一對多	<code>@OneToMany(mappedBy = "...")</code>	□	□	被控端，不需要 <code>@JoinColumn</code>
多對一	<code>@ManyToOne</code>	□	□	主控端，通常要加 <code>@JoinColumn</code>
一對一	<code>@OneToOne</code>	□	□	主控端加 <code>@JoinColumn</code> ; 雙向關聯可加 <code>mappedBy</code>
多對多	<code>@ManyToMany</code>	□ (使用 <code>@JoinTable</code>)	□	一定需要中介表 <code>@JoinTable</code>

12.1 JPA 關聯抓取資料策略

關聯類型	註解	預設抓取策略 (fetch)	常用替代值	建議說明與效能考量
一對一 (@OneToOne)	@OneToOne	EAGER (立即抓取)	LAZY	若非必要，建議設為 LAZY，避免不必要 JOIN
多對一 (@ManyToOne)	@ManyToOne	EAGER (立即抓取)	LAZY	預設 EAGER，但實務建議設成 LAZY，否則會造成大量 JOIN
一對多 (@OneToMany)	@OneToMany	LAZY (延遲抓取)	EAGER	預設安全；若設成 EAGER，需注意 N+1 查詢問題
多對多 (@ManyToMany)	@ManyToMany	LAZY (延遲抓取)	EAGER	若抓資料量大，建議保留 LAZY，EAGER 容易導致效能問題

關聯關係圖示

Author (1) ---- Book (N) --- publisher_book --- Publisher (N)
 |
 |
 Biography (1)

(仲介表)
(關聯表)

- Author 與 Book：一對多
- Book 與 Publisher：多對多
- Author 與 Biography：一對一

12.2 一對多關聯 (One-to-Many)

一位作者 (Author) 可以寫多本書 (Book) ，但每本書只能有一位作者。

這就是「一對多」的關聯：一個 Author 對應多個 Book 。

@Entity

```
public class Author {
```

@Id

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Integer id;
```

```
private String name;
```

```
@OneToMany(mappedBy = "author")
```

```
private List<Book> books;
```

```
}
```

說明：

1. @Entity：標示這是一個 JPA 實體，會對應到資料庫中的一個表格 (table) 。
2. @Id：主鍵欄位。
@GeneratedValue：主鍵自動生成策略。
3. private String name;：作者名字。
4. @OneToMany(mappedBy = "author")：
-> 表示 Author 和 Book 之間是一對多的關聯。
-> mappedBy = "author" 意思是這個關聯的「主控端」在 Book 實體的 author 屬性。
5. private List<Book> books;：一個作者擁有多本書。

@Entity

```
public class Book {
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Integer id;
```

```
private String name;
```

```
@ManyToOne
```

```
@JoinColumn(name = "author_id")
```

```
private Author author;
```

```
}
```

說明:

- @Entity : 標示這是一個 JPA 實體。
- @Id : 主鍵欄位。
- @GeneratedValue : 主鍵自動生成策略。
- private String name; : 書名。
- @ManyToOne :
表示多本書可以對應到同一位作者。
- @JoinColumn(name = "author_id") :
指定這個欄位 (author_id) 是外鍵，連接到 Author 的主鍵。
在資料庫的 Book 表格中會有一個 author_id 欄位，指向作者。

關聯關係圖示

Author (1) <----- (N) Book

一個 Author 可以有多本 Book。

每本 Book 只屬於一個 Author。

實際資料庫表格範例

Author 資料表

id	name
1	王小明

Book 資料表

id	name	author_id
1	Java 入門	1
2	Spring 實戰	1

12.3 一對一關聯 (One-to-One)

一位作者 (Author) 對應一份傳記 (Biography) ，每份傳記也只屬於一位作者。

這就是「一對一」的關聯：一個 Author 對應一個 Biography 。

@Entity

```
public class Author {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Integer id;
```

```
    private String name;
```

```
    @OneToMany(mappedBy = "author")
```

```
    private List<Book> books;
```

```
    @OneToOne(mappedBy = "author")
```

```
    private Biography biography;
```

```
}
```


說明:

1. @Entity : 標示這是一個 JPA 實體，對應資料庫中的一個表格 (table) 。
2. @Id : 主鍵欄位。
@GeneratedValue : 主鍵自動生成策略。
3. private String name; : 作者名字。
4. @OneToOne(mappedBy = "author") :
表示 Author 和 Biography 之間是一對一的關聯。
mappedBy = "author" 表示這個關聯的主控端在 Biography 實體的 author 屬性。
5. private Biography biography; : 一位作者對應一份傳記。

@Entity

```
public class Biography {
```

@Id

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Integer id;
```

```
private String details;
```

@OneToOne

```
@JoinColumn(name = "author_id")
```

```
private Author author;
```

```
}
```

說明:

- @Entity : 標示這是一個 JPA 實體。
- @Id : 主鍵欄位。
@GeneratedValue : 主鍵自動生成策略。
- private String details; : 傳記內容。
- @OneToOne :
表示 Biography 和 Author 之間是一對一的關聯。

@JoinColumn(name = "author_id") :

指定這個欄位 (author_id) 是外鍵，連接到 Author 的主鍵。

- 在資料庫的 Biography 表格中會有一個 author_id 欄位，指向作者。

關聯關係圖示

Author (1) <-----> (1) Biography

- 一位 Author 對應一份 Biography。
- 一份 Biography 只屬於一位 Author。

實際資料庫表格範例

Author 資料表

id	name
1	王小明

Biography 資料表

id	details	author_id
1	... 內容 ...	1

關聯端說明

- 在這個設計中，Biography 是主控端（因為它有 @JoinColumn）。
- mappedBy 屬性用來指明關聯的維護端在對方（這裡是 Biography 的 author 欄位）。
- 通常外鍵會放在其中一個表格（這裡放在 Biography 表格的 author_id 欄位）

12.4 多對多關聯 (Many-to-Many)

新增一個「出版社 (Publisher) 」實體，讓一本書可以由多個出版社發行，一個出版社也可以發行多本書，形成多對多關聯。

@Entity

```
public class Publisher {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Integer id;
```

```
    private String name;
```

```
    @ManyToMany
```

```
    @JoinTable(
```

```
        name = "publisher_book",
```

```
        joinColumns = @JoinColumn(name = "publisher_id"),
```

```
        inverseJoinColumns = @JoinColumn(name = "book_id")
```

```
    )
```

```
    private List<Book> books;
```

```
}
```

說明:

1. @ManyToMany：表示出版社和書籍之間是多對多關聯。
2. @JoinTable：指定中介表名稱與外鍵欄位名稱。

@Entity

```
public class Book {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Integer id;
```

```
private String name;
```

```
@ManyToOne
```

```
@JoinColumn(name = "author_id")
```

```
private Author author;
```

```
@ManyToMany(mappedBy = "books")
```

```
private List<Publisher> publishers;
```

```
}
```

說明:

@ManyToMany(mappedBy = "books")：代表多對多關聯的被控端，對應 Publisher 的 books 欄位。

中介表關聯說明

Publisher 和 Book 透過中介表 publisher_book 建立多對多關聯。

中介表 publisher_book 結構如下：

publisher_book 中介資料表

publisher_id	book_id
1	1
1	2
2	1

關聯關係圖示

Author (1) ----< Book (N) >--- publisher_book ----< Publisher (N)

|
|

Biography (1)

- Author 與 Book：一對多
- Book 與 Publisher：多對多
- Author 與 Biography：一對一

實際資料庫表格範例

Book 資料表

id	name	author_id
1	Java 入門	1
2	Spring 實戰	1

publisher_book 中介資料表

publisher_id	book_id
1	1
1	2
2	1

publisher 資料表

id	name
1	碁峰資訊
2	第三波資訊
3	天下文化

關聯端說明

- 這樣設計下，原本的 Author、Book、Biography 關聯不變。
- 多對多關聯只需在 Book 與 Publisher 之間新增關聯即可。

12.5 隨堂練習

執行前步驟:

- 1.刪除 book 資料庫
- 2.建立 book 資料庫
- 3.重新執行 springboot 專案 -> 會自動產生 tables
- 4.透過 Mysql workbench 執行以下 sql 語法

Sample SQL

```
INSERT INTO author (id, name) VALUES (1, '王大明');
INSERT INTO author (id, name) VALUES (2, '李小華');
INSERT INTO author (id, name) VALUES (3, '陳美玲');
INSERT INTO author (id, name) VALUES (4, '林志強');
INSERT INTO author (id, name) VALUES (5, '張育誠');

INSERT INTO book (id, name, author_id) VALUES (1, '從零開始學 C 語言', 1);
INSERT INTO book (id, name, author_id) VALUES (2, 'Python 程式設計入門', 2);
INSERT INTO book (id, name, author_id) VALUES (3, 'Java 核心技術實戰', 3);
INSERT INTO book (id, name, author_id) VALUES (4, '資料結構與演算法解析', 4);
INSERT INTO book (id, name, author_id) VALUES (5, '深入淺出 JavaScript', 5);
INSERT INTO book (id, name, author_id) VALUES (6, 'C 語言經典範例', 1);
INSERT INTO book (id, name, author_id) VALUES (7, 'Python 專案實作', 2);
INSERT INTO book (id, name, author_id) VALUES (8, 'Java 物件導向設計', 3);
INSERT INTO book (id, name, author_id) VALUES (9, '演算法設計與分析', 4);
INSERT INTO book (id, name, author_id) VALUES (10, 'JavaScript 網頁互動設計', 5);

INSERT INTO publisher (id, name) VALUES (1, '全華圖書');
INSERT INTO publisher (id, name) VALUES (2, '博碩文化');
INSERT INTO publisher (id, name) VALUES (3, '碁峰資訊');
INSERT INTO publisher (id, name) VALUES (4, '旗標出版');
INSERT INTO publisher (id, name) VALUES (5, '深智數位');

INSERT INTO biography (id, details, author_id) VALUES (1, '資深 C 語言講師，著有多本程式設計書籍。', 1);
INSERT INTO biography (id, details, author_id) VALUES (2, '專精 Python 與資料科學領域，經驗豐富。', 2);
INSERT INTO biography (id, details, author_id) VALUES (3, 'Java 技術專家，熱愛教學與寫作。', 3);
INSERT INTO biography (id, details, author_id) VALUES (4, '演算法與資料結構領域權威。', 4);
```

```
INSERT INTO biography (id, details, author_id) VALUES (5, '網頁前端開發達人 · 專攻 JavaScript', 5);
```

```
INSERT INTO publisher_book (publisher_id, book_id) VALUES (1, 1);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (2, 1);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (2, 2);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (3, 2);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (3, 3);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (4, 3);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (4, 4);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (5, 5);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (1, 6);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (2, 7);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (3, 8);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (4, 9);
INSERT INTO publisher_book (publisher_id, book_id) VALUES (5, 10);
```

5.查詢資料表內容

```
SELECT * FROM book.author;
SELECT * FROM book.book;
SELECT * FROM book.biography;
SELECT * FROM book.publisher;
SELECT * FROM book.publisher_book;
```

6.查詢問題:

- 一. 查詢每位作者的著作數量(列出作者姓名與書本數量)
- 二. 查詢同時由“全華圖書”與“博碩文化”出版且作者是“王大明”的書籍名稱
- 三. 查詢出版最多書籍的出版社名稱及其出版書本數量

參考答案

```
-- 一. 查詢每位作者的著作數量(列出作者姓名與書本數量)
select a.name, count(b.id) as book_count
from author a
left join book b on b.author_id = a.id
group by a.id, a.name;
```

-- 二. 查詢同時由“全華圖書”與“博碩文化”出版且作者是“王大明”的書籍名稱

```
select b.name
from book b
join author a on b.author_id = a.id
join publisher_book pb on pb.book_id = b.id
join publisher p on pb.publisher_id = p.id
where a.name = '王大明' and p.name in ('全華圖書', '博碩文化')
group by b.id, b.name
having count(DISTINCT p.name) = 2; -- 恰好有二家
```

-- 三. 查詢出版最多書籍的出版社名稱及其出版書本數量

```
select p.name, count(pb.book_id) as book_count
from publisher p
left join publisher_book pb on pb.publisher_id = p.id
group by p.id, p.name
order by book_count desc
limit 1
```


十三. 事務管理與資料庫交易

在 JPA 中，Spring 會自動處理事務。若沒有顯式指定，Spring 會在方法執行時自動開啟一個事務，並在方法結束時提交。如果有異常發生，Spring 會回滾事務。

[下載 Transaction PDF](#)

book 資料表內容

	book_id	book_name	book_price
	1	Java	70
▶	2	Spring	120
*	NULL	NULL	NULL

stock 資料表內容

	book_id	book_amount
	1	10
▶	2	10
*	NULL	NULL

wallet 資料表內容

	username	balance
▶	john	200
	mary	200
*	NULL	NULL

@Service

@Transactional

```
public class BookService {
```

@Autowired

```
private BookRepository bookRepository;
```

```
public void updateBook(Book book) {
```

```
    // 更新書籍
```

```
        bookRepository.save(book);  
    }  
}
```

@Transactional 註解可確保方法在一個事務內執行。

13.1 Propagation

Spring @Transactional 各種 Propagation 類型的詳細行為說明與實際範例

1. REQUIRED (預設)

行為說明：

- 若當前有交易，加入現有交易。
- 若無交易，自行開啟新交易。

適用情境：

- 一般業務邏輯，確保多個方法在同一交易範圍內執行，整體成功或失敗。

範例：

```
@Service
public class OrderService {
    @Transactional // 預設為 REQUIRED
    public void placeOrder() {
        createOrder(); // 會加入同一交易
        updateInventory();
    }

    @Transactional(propagation = Propagation.REQUIRED)
    public void createOrder() {
        // 寫入訂單資料
    }
}
```

placeOrder() 和 createOrder() 會共用同一個交易，任何一個失敗都會全部回滾

2. REQUIRES_NEW

行為說明：

- 總是開啟新交易，並暫停 (suspend) 外層現有交易，直到新交易結束。

適用情境：

- 需要獨立提交/回滾的操作，如審計、日誌、通知等，不希望受外層交易影響。

範例：

```
@Service
public class NotificationService {
    @Transactional
    public void sendMainProcess() {
        // 主交易
        sendNotification(); // 會啟動新交易
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void sendNotification() {
        // 發送通知，獨立交易
    }
}
```

若 sendNotification() 發生例外，只會回滾通知，不影響 sendMainProcess() 的主交易

3. NESTED

行為說明：

- 若有外層交易，則在現有交易內建立 Savepoint，失敗時只回滾到 Savepoint。
- 若無外層交易，等同於 REQUIRED。

適用情境：

- 需要局部回滾的複雜流程，如批次處理、部分失敗可容忍的業務，且底層資料庫支援 Savepoint。

範例：

```
@Service
public class BatchService {
    @Transactional
```

```
public void batchMain() {  
    processA();  
    try {  
        processB(); // 失敗只回滾 B  
    } catch (Exception e) {  
        // 處理異常，主交易繼續  
    }  
    processC();  
}  
  
@Transactional(propagation = Propagation.NESTED)  
public void processB() {  
    // 執行子流程，失敗時僅回滾本流程  
    if (someError) {  
        throw new RuntimeException("B 流程失敗");  
    }  
}  
}
```

processB() 發生錯誤，只回滾 B 內的操作，A 和 C 仍可繼續

機制差異

Propagation.REQUIRES_NEW 和 Propagation.NESTED 都用於處理方法巢狀呼叫時的交易行為，但它們的實現機制和適用場景有明顯差異：

特性	REQUIRES_NEW	NESTED
交易關係	完全獨立的新交易（掛起外層交易）	外層交易的子交易（同一個物理交易）
連線	使用新連線	使用同一連線
Savepoint	無，直接新開交易	有，建立 Savepoint（保存點）
影響	內外層互不影響，彼此獨立	子交易回滾不影響父交易，父交易回滾會一起回滾
多資料源支援	支援	需資料庫支援 Savepoint，JPA/Hibernate 通常不支援

4. SUPPORTS

行為說明：

- 有交易則加入，無交易則非交易方式執行。

適用情境：

- 可在有無交易環境下皆可安全執行的方法，如查詢、非關鍵性更新。

範例：

```
@Service
public class ProductService {
    @Transactional(propagation = Propagation.SUPPORTS)
    public Product getProduct(Long id) {
        // 查詢商品
        return productRepository.findById(id);
    }
}
```

```
}
```

若外層有交易則共用，否則直接查詢不開啟交易

5. NOT_SUPPORTED

行為說明：

- 若有現有交易，則暫停，非交易方式執行。

適用情境：

- 不希望在交易中執行的操作，如查詢、呼叫外部 API，避免鎖定資源。

範例：

```
@Service
public class ExternalApiService {
    @Transactional(propagation = Propagation.NOT_SUPPORTED)
    public void callExternalApi() {
        // 呼叫外部 API，不應該在交易中
    }
}
```

即使外層有交易，這段邏輯也會在非交易狀態下執行

6. NEVER

行為說明：

- 必須非交易方式執行，若有現有交易則拋出例外。

適用情境：

- 嚴格要求不能在交易中執行的操作，避免交易干擾。

範例：

```
@Service
public class ConfigService {
    @Transactional(propagation = Propagation.NEVER)
```

```
public void updateConfig() {  
    // 僅能在非交易狀態下執行  
}  
}
```

7. MANDATORY

行為說明：

- 必須有現有交易，否則拋出例外。

適用情境：

- 嚴格要求只能在交易上下文中執行的方法，如複雜的資料一致性操作。

範例：

```
@Service  
public class AuditService {  
    @Transactional(propagation = Propagation.MANDATORY)  
    public void writeAuditLog() {  
        // 必須在交易中執行  
    }  
}
```

若外層呼叫時沒有交易，會直接拋出例外

13.2 Isolation

1. 什麼是 Isolation ?

Isolation (隔離性) 是 ACID 四大交易特性之一，決定「一個交易中未提交的資料，其他交易是否能看到」，以及「多個交易同時操作同一筆資料時，會不會互相干擾」

2. 資料讀取問題

1. Dirty Read (髒讀)

定義：

一個交易讀取到另一個「尚未提交」的交易所修改過的資料。如果那個未提交的交易最後回滾，第一個交易讀到的就是不存在的「髒資料」。

範例：

- 交易 A 修改帳戶餘額，但尚未提交 (commit)。
 - 交易 B 此時讀取該帳戶餘額，看到的是交易 A 尚未提交的新值。
 - 如果交易 A 最後回滾，交易 B 其實讀到的是不存在的數據。
-

2. Non-repeatable Read (不可重複讀)

定義：

同一個交易在不同時間讀取同一筆資料時，因為其他交易已經提交了修改或刪除，導致兩次讀取結果不同。

範例：

- 交易 A 查詢商品價格，顯示\$50,000。
- 交易 B 在此期間將價格改為\$45,000 並提交。
- 交易 A 在同一交易內再次查詢，這次顯示\$45,000。
- 交易 A 在同一交易內查詢兩次，卻得到了不同的結果，這就是不可重複讀。

3. Phantom Read (幻讀)

定義：

同一個交易內，兩次查詢相同條件的多筆資料時，因為其他交易新增或刪除了符合條件的資料，導致查詢結果筆數不同。

範例：

- 交易 A 查詢所有特價商品，結果有 2 筆。
- 交易 B 在此期間新增一筆特價商品並提交。
- 交易 A 在同一交易內再次查詢，結果變成 3 筆。
- 交易 A 在同一交易內多次查詢，查到的資料筆數不一致，這就是幻讀。

現象	定義	範例簡述
髒讀 Dirty Read	讀到其他交易未提交的資料，該資料可能最後被回滾	交易 B 讀到交易 A 未 commit 的資料
不可重複讀 Non-repeatable Read	同一交易內兩次查同一筆資料，結果不同	交易 A 查兩次商品價格，結果不同
幻讀 Phantom Read	同一交易內兩次查詢多筆資料，筆數不同 (因其他交易新增/刪除資料)	交易 A 查詢商品清單，筆數變多

這三種現象是資料庫隔離級別設計時需要考慮的重要問題

3. Spring 支援的隔離層級 Isolation Levels

隔離級別	說明	會發生的問題
DEFAULT	使用底層資料庫預設隔離級別	依資料庫而異

READ_UNCOMMITTED	允許讀取未提交資料 (Dirty Read)	Dirty Read (髒讀) 、 Non-repeatable Read (不可重複讀) 、 Phantom Read (幻讀)
READ_COMMITTED	只能讀取已提交資料	Non-repeatable Read 、 Phantom Read
REPEATABLE_READ	一個交易內多次讀同一筆資料結果一致	Phantom Read
SERIALIZABLE	最嚴格，完全序列化執行，避免所有並發問題	無 (但效能最低)

各隔離級別詳細說明

1. DEFAULT

- 說明：採用資料庫預設隔離級別 (如 **MySQL 是 REPEATABLE_READ**，PostgreSQL 是 READ_COMMITTED) 。
- 建議：除非有特殊需求，通常建議使用 DEFAULT 。

2. READ_UNCOMMITTED

- 說明：允許交易讀到其他尚未提交的資料 (Dirty Read) 。
- 問題：可能讀到會被回滾的資料，會有 Dirty Read 、 Non-repeatable Read 、 Phantom Read 問題 。
- 適用：極少用於生產，僅適合對一致性要求極低、追求效能的場景 。

3. READ_COMMITTED

- 說明：只能讀到已經提交的資料，避免 Dirty Read 。
- 問題：仍可能發生 Non-repeatable Read 、 Phantom Read 。
- 適用：大部分商業系統常用隔離級別 (如 Oracle 、 PostgreSQL 預設) [156](#) 。

4. REPEATABLE_READ

- 說明：同一交易內多次查詢同一資料，結果一致，避免 Non-repeatable Read。
- 問題：仍可能有 Phantom Read。
- 適用：資料一致性要求較高的場景（MySQL 預設）[16](#)。

5. SERIALIZABLE

- 說明：最嚴格，所有交易完全序列化執行，避免所有並發問題。
- 問題：效能最低，並發度最差。
- 適用：對資料一致性要求極高的金融、帳務等場景 [146](#)。

4. 常見隔離級別與問題對照

隔離級別	Dirty Read	Non-repeatable Read	Phantom Read
READ_UNCOMMITTED	✓	✓	✓
READ_COMMITTED	✗	✓	✓
REPEATABLE_READ	✗	✗	✓
SERIALIZABLE	✗	✗	✗

十四. WebSocket

WebSocket 是一種網路通訊協議，提供雙向 (full-duplex)、即時、長連線的資料交換能力，運作於單一持久的 TCP 連線上。

這與傳統 HTTP 不同，HTTP 採用「請求-回應」模式，每次資料交換都要重新建立連線，而 WebSocket 只需在最初建立連線後，雙方即可隨時主動發送資料，連線會一直保持直到一方主動關閉。

14.1 WebSocket 的特點

- 雙向通訊 (Full-duplex)：用戶端和伺服器可同時主動發送訊息，無需等待對方請求。
- 即時性高：資料可即時推送，延遲極低，適合聊天室、即時通知、線上遊戲、股價看板等。
- 長連線：連線持續存在，直到任一方關閉，減少頻繁建立/關閉連線的資源消耗。
- 低協議負擔：連線升級後，資料以輕量封包 (frame) 傳遞，減少 HTTP header 的冗餘。

14.2 WebSocket 運作流程

1. HTTP 握手升級：用戶端發送 HTTP 請求，要求升級為 WebSocket 連線。
2. 協議升級：伺服器同意後回應 101 狀態，雙方進入 WebSocket 協議。
3. 持久連線：連線建立後，雙方可隨時主動發送訊息。
4. 關閉連線：任一方可隨時關閉連線，結束通訊 [210](#)。

14.3 應用場景

- 聊天室、即時通訊
- 線上多人遊戲

- 協同文件編輯
 - 即時數據儀表板、股價推播
 - IoT 裝置狀態同步
-

14.4 什麼是 STOMP ?

STOMP (Simple Text Oriented Messaging Protocol) 是一種簡單、文字導向的訊息協議，專為訊息代理 (Message Broker) 設計，常用於 WebSocket 上層協議。

1. STOMP 的特點

- 簡單易懂：協議設計類似 HTTP，易於解析與實作。
- 支援主題訂閱 (Publish/Subscribe)：用戶端可訂閱特定主題 (topic)，伺服器可將訊息廣播給所有訂閱者。
- 支援點對點 (Queue)：可實現一對一訊息傳遞 (如私訊)。
- 跨語言、跨平台：多種語言與平台皆有 STOMP 客戶端實作。

2. STOMP 運作流程

- 用戶端透過 **SEND** 指令發送訊息到指定目的地 (如 `/app/chat`)。
- 用戶端透過 **SUBSCRIBE** 指令訂閱主題 (如 `/topic/messages`)。
- 伺服器收到訊息後，根據目的地將訊息路由給對應的訂閱者。

3. STOMP + WebSocket

- STOMP 常作為 WebSocket 的應用層協議，讓訊息交換更有結構，易於管理與擴展。
- 例如：聊天室訊息廣播、用戶狀態通知等。

項目	WebSocket	STOMP (通常運作於 WebSocket 之上)
定位	傳輸協議 (Transport Protocol)	應用層協議 (Messaging Protocol/Subprotocol)
主要功能	提供雙向、即時、長連線的資料傳輸通道	定義訊息格式與語意，支援主題訂閱、訊息廣播等
訊息格式	不結構化 (純文字或二進位，內容自訂)	結構化 (簡單文字協議，類似 HTTP header + body)
典型用途	任意即時應用 (如遊戲、IoT、聊天室、通知)	訊息導向應用 (如聊天室、通知、即時協作、消息中介)
協議特性	只負責通訊管道，不處理訊息語意	提供 SEND、SUBSCRIBE、UNSUBSCRIBE、MESSAGE 等語意
擴充性	需自行設計訊息格式與協議	提供標準訊息結構，跨語言、跨平台兼容
與 Spring 整合	可用低階 Handler (TextWebSocketHandler)	用高階 STOMP Controller (@MessageMapping)
典型 JavaScript 寫法	<code>new WebSocket(url)</code> ，onmessage 收發資料	用 STOMP 客戶端 (如 stompjs) 連線、訂閱、發送
互動模式	任意訊息，需自行處理路由與協議	支援主題 (topic) 訂閱、訊息廣播、ACK、事務等
可讀性	訊息內容自訂，不易直接閱讀	訊息為純文字格式，易於除錯與分析

項目	WebSocket	STOMP (通常運作於 WebSocket 之上)
定位	傳輸協議 (Transport Protocol)	應用層協議 (Messaging Protocol/Subprotocol)
依賴關係	直接運作於 TCP (經 HTTP 升級)	需依賴底層傳輸協議 (如 WebSocket 、 TCP 、 HTTP 等)
適合對象	需自訂協議或極致效能的應用	需快速開發、標準化、跨語言訊息互通的應用

簡單比喻：

WebSocket 像是電話線路，STOMP 則像是雙方約定的通話規則和語言；有了規則，溝通才有秩序，否則只能亂講話。

選用建議：

- 若需快速開發、訊息結構化、多人廣播、跨語言整合，建議用 STOMP over WebSocket。
- 若需極致效能、協議自訂，或只需點對點通訊，可直接用 WebSocket。

14.5 Spring Boot 的 WebSocket 支援

Spring Boot 提供兩種主要 WebSocket 實作方式：

1. 高階寫法 (STOMP Controller)

- 使用 `@EnableWebSocketMessageBroker` 啟用 STOMP 支援。
- 以 `@Controller`、`@MessageMapping`、`@SendTo` 等註解處理訊息。

- Spring 內建簡單訊息代理 (Simple Broker) ，也可整合外部訊息代理 (如 RabbitMQ) 。
- 適合大多數業務應用，開發簡單、易於擴充與維護。

2. 低階寫法 (Handler)

- 使用 `@EnableWebSocket` 及實作 `WebSocketHandler` 或 `TextWebSocketHandler` 。
- 需手動處理連線、訊息收發、序列化等細節。
- 適合自訂協議或特殊需求。

14.6 Spring Boot WebSocket 架構優勢

- 自動配置：`spring-boot-starter-websocket` 依賴即可自動啟用 WebSocket 支援。
- 與 MVC 整合：可與 Spring Security、驗證、資料綁定等功能整合。
- 開發快速：高階寫法大幅簡化開發流程，降低出錯機率。

14.7 實作練習

專案依賴

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

專案結構

```
src
├── main
│   ├── java
│   │   ├── com.example.demo
│   │   │   ├── ChatController.java
│   │   │   ├── WebSocketConfig.java
│   │   │   ├── ChatMessage.java
│   │   │   └── WebSocketEventListener.java
│   └── resources
│       ├── static
│       └── chat.html
```

ChatController.java

- 角色：WebSocket 訊息的主要處理器。
- 功能：用 @Controller、@RequestMapping 處理來自前端的聊天訊息，並用 @SendTo 廣播給所有訂閱者。
- 說明：這是 STOMP 架構下的高階 Controller，讓你能用物件（如 ChatMessage）直接收發訊息，無需處理底層細節。

- WebSocketConfig.java

- 角色：WebSocket 與 STOMP 的組態設定檔。
- 功能：設定 WebSocket 端點（如 /chat-websocket）、STOMP 主題（如 /topic）、應用前綴（如 /app）。
- 說明：用 @EnableWebSocketMessageBroker 啟用 STOMP 訊息代理，讓你的應用支援主題訂閱、廣播等功能。

- ChatMessage.java

- 角色：訊息資料模型（Model）。
- 功能：定義聊天訊息的資料結構（如 from、content），前後端都用這個格式進行資料交換。
- 說明：讓 Spring 自動將前端傳來的 JSON 轉成 Java 物件，回傳時也自動轉成 JSON。

- WebSocketEventListener.java

- 角色：WebSocket 事件監聽器。

- 功能：監聽用戶連線 (Connect)、斷線 (Disconnect) 等事件，可用於記錄日誌、管理用戶狀態、廣播系統訊息。
 - 說明：用 @EventListener 搭配 Spring 的事件類別 (如 SessionConnectEvent、SessionDisconnectEvent)，讓你能即時掌握用戶進出聊天室的狀態。
-

2. src/main/resources/static/

這是靜態資源目錄，所有放在這裡的檔案 (如 HTML、JS、CSS) 都能直接被瀏覽器存取。

- chat.html

- 角色：聊天室前端頁面。
 - 功能：提供用戶輸入暱稱、訊息，並即時顯示所有人的聊天內容。
 - 說明：內含 JavaScript 程式，透過 SockJS + STOMP 連線到後端 WebSocket，收發訊息並更新畫面。
-

3. 運作流程總覽

1. 用戶開啟 chat.html，前端 JS 透過 SockJS/STOMP 連線 /chat-websocket。
2. 用戶輸入訊息並送出，JS 以 JSON 格式送到 /app/chat。
3. ChatController 收到訊息，廣播到 /topic/messages。
4. 所有訂閱 /topic/messages 的用戶 (前端) 即時收到訊息並顯示。
5. WebSocketEventListener 監聽用戶進出，便於管理用戶狀態或廣播系統訊息。

十五. 隨堂練習：Spring Boot WebSocket 多聊天室與廣播

練習目標

1. 熟悉 Spring Boot WebSocket 與 STOMP 訊息協定的整合。
 2. 學會設計多聊天室系統，並能動態根據聊天室 ID 路由訊息。
 3. 能夠實作全域廣播功能，讓所有聊天室同時收到特定訊息。
-

題目說明

請根據下列需求，設計並實作一個簡易的多聊天室聊天室系統，並加入「全域廣播」功能。

需求一：多聊天室聊天

- 使用者可以從前端介面選擇不同聊天室（如聊天室 1、聊天室 2、聊天室 3）。
- 不同聊天室的訊息應該彼此隔離，只有進入同一聊天室的使用者才能看到對方訊息。
- 前端頁面需有聊天室選單、訊息顯示區、輸入框與送出按鈕。

需求二：聊天室 3 自動推送時間

- 當使用者選擇聊天室 3 時，伺服器每秒自動推送目前時刻訊息到聊天室 3，所有在聊天室 3 的使用者都能即時看到。
- 將訊息直接發送到 `"/topic/messages/room3"`

需求三：全域廣播

- 前端需有一個「廣播所有聊天室」按鈕。
- 按下此按鈕後，發送的訊息會同時顯示在所有聊天室，不論使用者目前在哪個聊天室。
- 廣播訊息在畫面上需有明顯標示（如紅色或加粗）。

加分挑戰（選做）

- 廣播訊息只允許特定身份（如管理員）發送。
 - 聊天室可動態新增（讓使用者自行輸入聊天室名稱）。
 - 廣播訊息可區分型態（如系統公告、活動推播等）。
-

參考提示

- 後端請使用 Spring Boot + WebSocket + STOMP，並用 `SimpMessagingTemplate` 動態推送訊息。
- 前端建議使用原生 HTML/JS 搭配 sockjs-client 與 stomp.js。
- 廣播功能可設計一個固定 topic (如 `/topic/broadcast`)，所有聊天室都要訂閱這個 topic。
- 時間推送可用 Spring 的 `@Scheduled` 定時任務實現。

Tomcat SSL



1. 產生自簽名憑證

使用 JDK 內建的 `keytool` 工具產生 PKCS12 格式的金鑰庫 (`keystore`) :

```
keytool -genkeypair -alias tomcat -storetype PKCS12 -keyalg RSA -keysize  
2048 -keystore keystore.p12 -validity 3650 -ext san=ip:127.0.0.1
```

- `-alias tomcat` : 憑證別名
- `-storetype PKCS12` : 金鑰庫格式 (推薦 PKCS12)
- `-keyalg RSA` : 加密演算法
- `-keysize 2048` : 金鑰長度
- `-keystore keystore.p12` : 輸出的金鑰庫檔案名稱
- `-validity 3650` : 有效天數 (例如 10 年)
- `-ext san=ip:127.0.0.1` : Subject Alternative Name , 建議填寫實際開發用 IP 或網域

2. 設定 Spring Boot

將 `keystore.p12` 放到 `src/main/resources` 目錄下 , 然後在 `application.properties` (或 `application.yml`) 中加入以下設定 :

```
server.port=8443  
server.ssl.key-store=classpath:keystore.p12  
server.ssl.key-store-password=你的密碼  
server.ssl.key-store-type=PKCS12  
server.ssl.key-alias=tomcat
```

- `server.port` : HTTPS 監聽的埠號 (常用 8443)

- `server.ssl.key-store`：金鑰庫路徑
- `server.ssl.key-store-password`：建立憑證時設定的密碼
- `server.ssl.key-store-type`：金鑰庫型態
- `server.ssl.key-alias`：憑證別名

3. 啟動與測試

啟動 Spring Boot 專案後，使用 `https://localhost:8443` 存取。若用自簽名憑證，瀏覽器會提示「不受信任」，開發時可忽略。

4. 讓 Insomnia 可以連到自簽的 HTTPS 網站

SpringSecurity



一、運作架構與流程

1. 運作架構概述

- Spring Security 以一系列 Filter (過濾器) 為核心，攔截所有 HTTP 請求，進行認證與授權。
- Filter 由 Servlet 的 DelegatingFilterProxy 代理，串接多個 Spring Security Filter，與 Spring MVC 無縫整合。

2. 表單登入流程

1. HTTP 請求進入：進入 Servlet FilterChain，由 DelegatingFilterProxy 導向 Spring Security FilterChain。
2. SecurityContextPersistenceFilter：載入/儲存當前用戶的安全上下文 (SecurityContext)。
3. UsernamePasswordAuthenticationFilter：攔截登入表單，驗證帳號密碼。
4. 認證成功/失敗處理：成功則將資訊存入 SecurityContextHolder，失敗則返回登入頁。
5. ExceptionTranslationFilter：處理認證/授權異常。
6. FilterSecurityInterceptor：根據權限規則授權資源存取。
7. SecurityContextHolder：管理當前執行緒的安全上下文。

二、主要組件說明

組件名稱	角色與功能
DelegatingFilterProxy	Servlet Filter 與 Spring Bean 橋樑，導向 Security

	FilterChain
FilterChainProxy	管理多個 Spring Security Filter 的鏈結與調用順序
SecurityContextPersistenceFilter	載入/儲存 SecurityContext (用戶認證資訊)
UsernamePasswordAuthentication Filter	處理表單登入請求，執行用戶認證
ExceptionTranslationFilter	處理認證/授權異常，決定是否跳轉登入頁或回應錯誤
FilterSecurityInterceptor	根據認證資訊與權限規則進行資源授權判斷
SecurityContextHolder	存取當前執行緒的安全上下文，保存認證資訊

三、完整 Java 配置範例

```
demo/  
├─ src/  
│   └─ main/  
│       └─ java/  
│           └─ com/
```

```
|  |  └─ example/
|  |    └─ demo/
|  |      └─ config/
|  |        └─ SecurityConfig.java
|  |          └─ controller/
|  |            └─ AdminController.java
|  |            └─ ErrorController.java
|  |            └─ LoginController.java
|  |            └─ UserController.java
|  └─ resources/
|    └─ static/
|      └─ index.html
|    └─ templates/
|      └─ accessDenied.html
|      └─ login.html
└─ pom.xml
```

- Java 檔案放在 `src/main/java/com/example/demo/`
- Thymeleaf 網頁放在 `src/main/resources/templates/`
- 靜態資源 (html) 放在 `src/main/resources/static/`
- 請參考範例程式

React



模組 1：React 基礎概念

1.1 React 環境設置與專案初始化

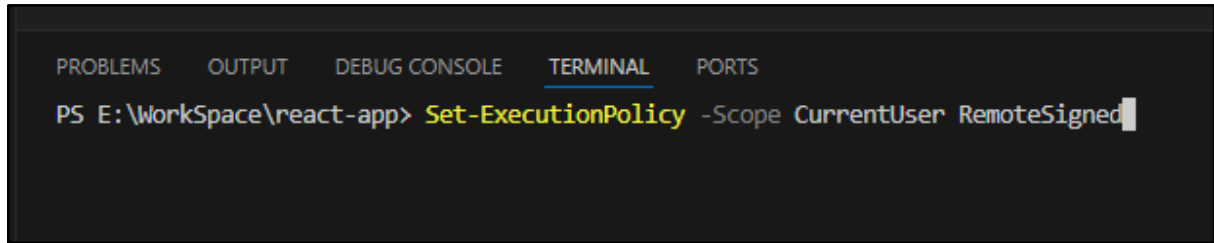
- 學習目標：了解 React 開發環境的安裝和設置，學會使用 Vite 快速初始化一個 React 專案。
- 內容說明：
 - 安裝 Node.js 和 npm
<https://nodejs.org/zh-tw/download/prebuilt-installer>
 - 開發環境 VSCode。
<https://vscode.dev.org.tw/download>
 - 使用 Vite 建立 React 專案，介紹 Vite 的優勢及其在 React 開發中的應用。
 - 指令: `npm create vite@5.2.2 // 指定版本`
 - 指令: `npm create vite@latest // 最新版本`

強迫開放權限

VSCode 執行終端機(以系統管理員身分開啟 VSCode)

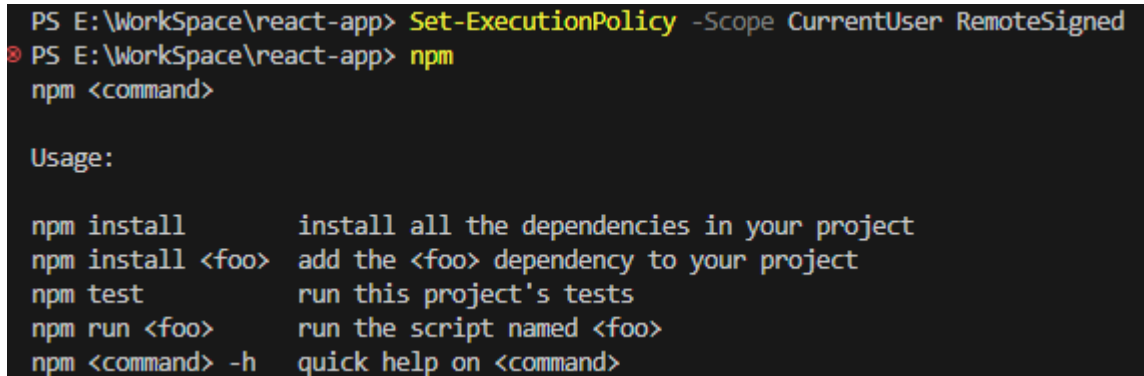
輸入:

Set-ExecutionPolicy -Scope CurrentUser RemoteSigned



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\WorkSpace\react-app> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

測試輸入: npm



```
PS E:\WorkSpace\react-app> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
PS E:\WorkSpace\react-app> npm
npm <command>

Usage:

npm install      install all the dependencies in your project
npm install <foo> add the <foo> dependency to your project
npm test         run this project's tests
npm run <foo>    run the script named <foo>
npm <command> -h quick help on <command>
```

成功~~

或參考 <https://akoncc.github.io/2019/11/01/vscode-cant-run-script/>

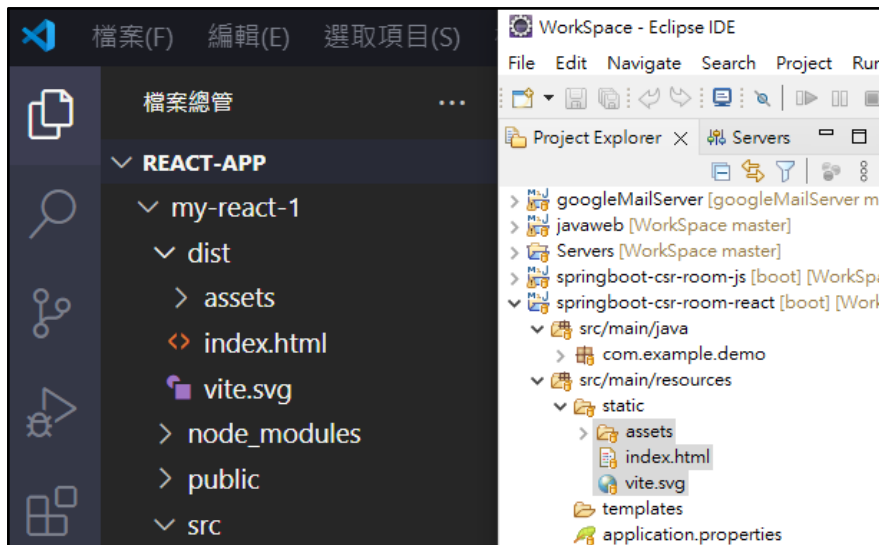
Windows PowerShell 基本操作 - 執行 Windows PowerShell 腳本

<https://ithelp.ithome.com.tw/articles/10028377>

打包 React 專案

指令: npm run build

將 vscode 專案的 /dist 資料複製到 springboot 專案下的 /src/main/resources/static



1.2 JSX 與元件基礎

- 學習目標：理解 JSX 語法及 React 元件的基本概念。
- 內容說明：
 - 介紹 JSX 語法，解釋 JSX 如何在 React 中被轉譯為 JavaScript。
 - 建立簡單的 React 元件並嵌入 JSX，展示元件的基本組成。
 - 示範基本的 HTML 標籤和 JSX 表達式嵌入（例如 `{}` 語法）。
 - 解釋如何使用 JSX 簡化代碼並提升元件的可讀性。

1.3 元件組合與屬性傳遞

- 學習目標：理解元件之間的組合與 `props` 屬性傳遞。
- 內容說明：
 - 建立多個元件，並展示如何在父元件中嵌入子元件。
 - 使用 `props` 向子元件傳遞數據，展示 `props` 在元件之間的通訊功能。
 - 示範將屬性（如 `x`、`y`）傳遞給 `MyComponent1`，並在子元件中使用。

模組 2：狀態管理與事件處理

2.1 使用 `useState` 管理狀態

- 學習目標：掌握 `useState` 的用法，理解 React 如何管理元件的狀態。
- 內容說明：

- 介紹 `useState`，並示範如何創建狀態變數（如 `message`）。
- 說明狀態更新和元件重新渲染的原理。
- 使用 `setMessage` 更新 `message` 的值，並展示 UI 如何根據狀態變化即時更新。

2.2 事件處理

- 學習目標：學會處理元件中的事件並觸發狀態變化。
- 內容說明：
 - 說明如何將事件（如 `onClick` 或 `onChange`）綁定到按鈕或其他元素。
 - 示範點擊按鈕後如何觸發 `setMessage` 更新狀態。
 - 在 `MyInput` 中建立訊息輸入表單，實現新增訊息功能。
 - 在 `MyDisplay` 中呈現訊息功能。

案例練習

1. 利用 DOM 的操作方式進行渲染

```
import './App.css'

// var: 作用範圍 function scope, 可以多次重複宣告
// let: 作用範圍 block scope, 同一個 scope 不可重複宣告
// const: 作用範圍 block scope, 不可重複宣告

function App() {
  let message = "";

  const handleChange = (e) => {
    message = e.target.value; // 變數改變, 但是畫面不會渲染更新
    console.log('message:', message);

    // 直接用 DOM 操作來更新內容, 如此渲染更新
    document.getElementById('displayMessage').textContent = `顯示: ${message}`;
  };

  return (
    <>
    <div>
      <input type="text" placeholder="請輸入一些內容" onChange={handleChange} />
    </div>
    <div id="displayMessage">
      顯示: {message}
    </div>
    </>
  )
}
```

```
export default App
```

2. 利用 React 所提供的 Hook: useState

```
import { useState } from 'react';
import './App.css'

function App() {
  const [message, setMessage] = useState(""); // '' 表示 message 的初始值

  const handleChange = (e) => {
    setMessage(e.target.value); // 透過 setMessage 變更 message 變數內容後網頁會自動渲染更新
    console.log('message:', message);
  };

  return (
    <>
      <div>
        <input type="text" placeholder="請輸入一些內容" onChange={handleChange} />
      </div>
      <div>
        顯示: {message}
      </div>
    </>
  )
}

export default App
```

3. 進行組件化

```
return (
  <>
    <div>
      <input type="text" placeholder='請輸入一些內容' onChange={handleChange} />
    </div>
    <div>
      顯示: {message}
    </div>
  </>
)
```

變更為組件化

```
return (
  <>
    <MyInput setMessage={setMessage} />
    <MyDisplay message={message} />
  </>
)
```

完整程式碼

```
import { useState } from 'react';
import './App.css'

const MyInput = ({setMessage}) => {

  const handleChange = (e) => {
    setMessage(e.target.value); // 透過 setMessage 變更 message 變數內容後網頁會自動渲染更新
    console.log('e.target.value:', e.target.value);
  };

  return (
    <div>
```

```
<input type="text" placeholder='請輸入一些內容' onChange={handleChange} />
</div>
);
};

const MyDisplay = ({message}) => {
  return (
    <div>
      顯示: {message}
    </div>
  );
};

function App() {
  const [message, setMessage] = useState(""); // " 表示 message 的初始值

  return (
    <>
      <MyInput setMessage={setMessage} />
      <MyDisplay message={message} />
    </>
  )
}

export default App
```

4. 進行組件檔案分離

將 `<MyInput>` 與 `<MyDisplay>` 分開成二個 `jsx` 文件

例如: `MyInput.jsx` 與 `MyDisplay.jsx`

放在 `components` 目錄下

src

```
|— components
|   |— MyInput.jsx
|   |— MyDisplay.jsx
|— App4.jsx
```

`components/MyInput.jsx`

```
import React from "react";

const MyInput = ({setMessage}) => {

  const handleChange = (e) => {

    setMessage(e.target.value); // 透過 setMessage 變更 message 變數內容後網頁會自動渲染更新
    console.log("e.target.value:", e.target.value);
  };

  return (
    <div>
      <input type="text" placeholder='請輸入一些內容' onChange={handleChange} />
    </div>
  );
};

export default MyInput;
```

components/MyDisplay.jsx

```
import React from "react";

const MyDisplay = ({message}) => {
  return (
    <div>
      顯示: {message}
    </div>
  );
};

export default MyDisplay;
```

App4.jsx

```
import { useState } from 'react';
import './App.css'
import MyInput from './components/MyInput';
import MyDisplay from './components/MyDisplay';

function App() {
  const [message, setMessage] = useState(""); // " 表示 message 的初始值

  return (
    <>
      <MyInput setMessage={setMessage} />
      <MyDisplay message={message} />
    </>
  )
}

export default App
```

執行結果

顯示: Hello React

模組 3：使用 `useEffect` 處理副作用

● 3.1 基本使用 `useEffect` 處理副作用

- 學習目標：理解 `useEffect` 的作用，學會在元件中處理副作用。
- 內容說明：
 - 介紹 `useEffect` 的基本用法，理解什麼是副作用（如資料獲取、DOM 操作）。
 - 使用 `useEffect` 模擬初次渲染時的資料抓取。
 - 在 `ScoreList2` 中利用 `useEffect` 模擬從伺服器獲取分數資料，並將資料設置到狀態中。

● 3.2 動態依賴的 `useEffect`

- 學習目標：掌握依賴變量的作用，學會控制 `useEffect` 的執行時機。
- 內容說明：
 - 在 `ExampleComponent` 中，使用 `count` 作為 `useEffect` 的依賴變量，讓資料隨 `count` 變化重新獲取。
 - 說明依賴陣列如何影響 `useEffect` 的執行。
 - 使用 `setTimeout` 模擬網路請求延遲，體驗真實應用中的非同步請求。

模組 4：清單渲染與條件顯示

● 4.1 使用 `map` 渲染清單

- 學習目標：掌握 `map` 函數用於渲染清單的用法。
- 內容說明：
 - 使用 `map` 函數在 `MyMessageList` 中渲染訊息清單，展示如何動態生成清單項目。
 - 說明 `key` 屬性的用途，並解釋為什麼需要為每個列表項指定唯一 `key` 值。
 - 展示 `ScoreList1` 中根據 `scores` 陣列渲染分數清單。

● 4.2 條件渲染

- 學習目標：學會在 React 中進行條件渲染。
- 內容說明：
 - 根據 `showPassing` 狀態控制顯示及格或不及格的分數列表。

- 使用三元運算子或邏輯運算符進行條件判斷。
- 結合 `ScoreList1` 中的分數列表，展示如何切換顯示不同條件的資料。

模組 5：進階應用

● 5.1 多組狀態管理

- 學習目標：學會使用多組 `useState` 來管理多組資料。
- 內容說明：
 - 在 `ScoreList3` 中使用 `useState` 同時管理 `scores`、`passingScores` 和 `failingScores`。
 - 說明如何將多個 `useState` 狀態結合在一個元件中使用。

● 5.2 回呼函數狀態更新

- 學習目標：掌握在 `setState` 中使用回呼函數更新狀態的技巧。
- 內容說明：
 - 說明如何使用 `(prevScores) => [...prevScores, newScore]` 格式的回呼函數來避免狀態更新的競爭條件。
 - 在 `ScoreList3` 的 `addRandomScore` 函數中使用回呼函數，展示如何動態添加新數據。

● 5.3 動態生成數據

- 學習目標：學會動態生成數據並新增到狀態中。
- 內容說明：
 - 在 `ScoreList3` 中使用 `Math.random()` 隨機生成分數和 ID。
 - 使用 `addRandomScore` 函數動態生成分數數據，並即時更新顯示。

模組 6：資料展示與格式化

● 6.1 使用 JSON 格式展示資料

- 學習目標：掌握如何格式化數據並顯示於元件中。
- 內容說明：
 - 使用 `JSON.stringify` 將資料轉換為可視化的 JSON 格式。
 - 在 `ExampleComponent` 中顯示 JSON 格式的 `data`，讓學生了解如何格式化 API 回應的資料。

● 6.2 基礎樣式設計

- 學習目標：學會簡單使用行內樣式管理元件的佈局。
- 內容說明：
 - 在 `ScoreList3` 中使用 `display: 'flex'` 設計佈局，讓及格與不及格的分數列表並排顯示。
 - 說明 CSS 行內樣式的使用方法，並展示如何控制元件的基本樣式。

模組 7：課程綜合實作

● 7.1 React 應用程式開發實作

- 學習目標：綜合應用所學知識，開發一個完整的小型 React 應用。
- 內容說明：
 - 開發一個分數管理系統，學生可以動態新增訊息、管理分數列表、根據條件顯示資料。
 - 綜合使用 `useState`、`useEffect`、條件渲染、清單渲染等技術完成應用。

● 7.2 課程回顧與答疑

- 學習目標：回顧課程內容，加深對核心概念的理解，並解答學生的疑問。
- 內容說明：
 - 課程知識點總結，包括 `useState`、`useEffect`、事件處理、清單渲染等。
 - 回答學生在課程中遇到的問題，並提供實踐建議和進一步學習資源。

模組 8：React 打包指令與步驟

建立一個 springboot-client 專案

將寫好的 react client 程式打包到 springboot

1. `npm run build`
2. 進入 `dist` 目錄, 並將 `dist` 目錄下的檔案與目錄(不含 `dist`)全部複製到 `springboot-client` 專案 `src/main/resources/static` 目錄下

TodoList 範例



TodoList 與 Json 範例:

My Todo List

吃早餐	☒
早晨運動 - 跑步 30 分鐘	☐
閱讀技術書籍 - 每日 20 頁	☒
複習昨天的程式練習	☐
參加線上 React 課程	☐
完成 React To-Do List 專案練習	☐
中午休息，午睡	☐
下午進行專案開發 - 完成登入功能	☐
更新學習日記	☐
晚上與朋友一起聚餐	☐

```

"todolist": [
  {
    "id": "1",
    "text": "吃早餐",
    "completed": true
  },
  {
    "id": "2",
    "text": "早晨運動 - 跑步 30 分鐘",
    "completed": false
  },
  {
    "id": "3",
    "text": "閱讀技術書籍 - 每日 20 頁",
    "completed": true
  },
  {
    "id": "4",
    "text": "複習昨天的程式練習",
    "completed": false
  },
  {
    "id": "5",
    "text": "參加線上 React 課程",
    "completed": false
  },
  {
    "id": "6",
    "text": "完成 React To-Do List 專案練習",
    "completed": false
  },
  {
    "id": "7",
    "text": "中午休息，午睡",
    "completed": false
  },
  {
    "id": "8",
    "text": "下午進行專案開發 - 完成登入功能",
    "completed": false
  },
  {
    "id": "9",
    "text": "更新學習日記",
    "completed": false
  }
]

```

Java 企業框架實務班課程講-TodoList 範例

	<pre>}, { "id": "10", "text": "晚上與朋友一起聚餐", "completed": false }]</pre>
--	---

撰寫演進步驟:

App.jsx 基礎樣式

```
import './App.css'

function App() {

  return (

    <>

      <h1>My Todo List</h1>

      <div>

        <input type="text" />

        <button>Add</button>

      </div>

      <ul>

        <li>吃早餐</li>

        <li>做運動</li>

        <li>寫程式</li>

      </ul>

    </>

  )
}

export default App
```

My Todo List

Add

- 吃早餐
- 做運動
- 寫程式

App2.jsx 加入 todos 變數

```
import './App.css'

function App() {

  const todos = [
    '吃早餐', '做運動', '寫程式', 'Debug'
  ]

  return (
    <>
      <h1>My Todo List</h1>
      <div>
        <input type="text" />
        <button>Add</button>
      </div>
      <ul>
        {
```



```
    todos.map((todo, index) => {  
      return (<li key={index}>{todo}</li>)  
    })  
  }  
</ul>  
</>  
)  
}  
  
export default App
```

簡化寫法:

```
{  
  todos.map((todo, index) => {  
    return (<li key={index}>{todo}</li>)  
  })  
}
```

簡化寫法:

```
{todos.map((todo, index) => (<li key={index}>{todo}</li>))}
```

App3.jsx 加入 useState()

```
import { useState } from 'react'  
import './App.css'
```

```
function App() {

  const [todos, setTodos] = useState([
    '吃早餐', '做運動', '寫程式', 'Debug'
  ]);

  const [todo, setTodo] = useState("");

  const handleClick = (e) => {
    if(!todo) return;
    //setTodos(todos.concat(todo));
    setTodos([...todos, todo]);
    setTodo(""); // 將 todo 清空
  };

  const handleChange = (e) => {
    setTodo(e.target.value);
  };

  return (
    <>
      <h1>My Todo List</h1>
      <div>
        <input type="text" onChange={handleChange} value={todo} />
        <button onClick={handleClick}>Add</button>
      </div>
      <ul>
```

```
      {todos.map((todo, index) => (<li key={index>{todo</li>)))}

    </ul>

  </>

)

}

export default App
```

App4.jsx 設計 todo 資料結構

設計 todo 資料結構 `{id: 1, text: '吃早餐', completed: true}`

加上 checkbox

todo.id 替代 index

```
import { useState } from 'react'
import './App.css'

function App() {

  const [todos, setTodos] = useState([
    {id: 1, text: '吃早餐', completed: true},
    {id: 2, text: '做運動', completed: false},
    {id: 3, text: '寫程式', completed: true},
    {id: 4, text: 'Debug', completed: false},
  ]);

  const [todo, setTodo] = useState("");

  const handleClick = (e) => {
```

```
if(!todo) return;

// 取得 id 值 = 目前最大 id + 1

const newId = todos.length > 0 ? Math.max(...todos.map((t) => t.id)) + 1 : 1;

const newTodo = {
  id: newId, text: todo, completed: false
};

setTodos([...todos, newTodo]);

setTodo(""); // 將 todo 清空
};

const handleChange = (e) => {
  setTodo(e.target.value);
};

const toggleCompletion = (id) => {
  setTodos(
    todos.map((todo) => todo.id === id ? {...todo, completed: !todo.completed} : todo)
  );
};

return (
  <>
    <h1>My Todo List</h1>
    <div>
      <input type="text" onChange={handleChange} value={todo} />
      <button onClick={handleClick}>Add</button>
    </div>
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>
```

```
      {todo.id}

      <input type="checkbox" onChange={() => toggleCompletion(todo.id)}
checked={todo.completed} />

      {todo.text}
    </li>
  )))
</ul>
</>
)
}

export default App
```

App5.jsx 切模組

src

```
├── components
│   ├── pure
│   │   ├── TodoInput.jsx
│   │   ├── TodoItem.jsx
│   │   └── TodoList.jsx
└── App5.jsx
```

components/TodoInput.jsx

```
import React from "react";

const TodoInput = ({todo, onChange, onAdd}) => {
```

```
return (  
  <div>  
    <input type="text" onChange={onChange} value={todo} />  
    <button onClick={onAdd}>Add</button>  
  </div>  
)  
  
}  
  
export default TodoInput;
```

components/TodoItem.jsx

```
import React from 'react';  
  
const TodoItem = ({ todo, onToggleCompletion }) => {  
  return (  
    <li>  
      {todo.id}  
      <input  
        type="checkbox"  
        checked={todo.completed}  
        onChange={() => onToggleCompletion(todo.id)}  
      />  
      {todo.text}  
    </li>  
  )  
}
```

```
);  
};  
  
export default TodoItem;
```

components/ToDoList.jsx

```
import React from 'react';  
import TodoItem from './TodoItem';  
  
const TodoList = ({ todos, onToggleCompletion }) => {  
  return (  
    <ul>  
      {todos.map((todo) => (  
        <TodoItem  
          key={todo.id}  
          todo={todo}  
          onToggleCompletion={onToggleCompletion}  
        />  
      ))}  
    </ul>  
  );  
};  
  
export default TodoList;
```

App5.jsx

```
import { useState } from 'react';
import './App.css';
import TodoList from './components/TodoList';
import TodoInput from './components/TodoInput';

function App() {
  const [todos, setTodos] = useState([
    { id: 1, text: '吃早餐', completed: true },
    { id: 2, text: '做運動', completed: true },
    { id: 3, text: '寫程式', completed: true },
    { id: 4, text: 'Debug', completed: false },
  ]);

  const [todo, setTodo] = useState("");

  const handleAdd = () => {
    if (!todo) return;
    const newId = todos.length > 0 ? Math.max(...todos.map((t) => t.id)) + 1 : 1;
    const newTodo = { id: newId, text: todo, completed: false };
    setTodos([...todos, newTodo]);
    setTodo("");
  };

  const handleChange = (e) => {
    setTodo(e.target.value);
  };

  const toggleCompletion = (id) => {
    setTodos(
      todos.map((todo) =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      )
    );
  };
}
```



```
);  
};  
  
return (  
  <div>  
    <h1>My Todo List</h1>  
    <TodoInput todo={todo} onChange={handleChange} onAdd={handleAdd} />  
    <TodoList todos={todos} onToggleCompletion={toggleCompletion} />  
  </div>  
);  
}  
  
export default App;
```

加入 bootstrap 美化

Bootstrap 是一個功能強大的前端工具組，旨在協助開發者快速建立響應式、行動裝置優先的網站。

[Bootstrap 下載站](#)

透過提供一系列預先設計的 HTML、CSS 和 JavaScript 元件，Bootstrap 簡化了網頁開發流程，確保在不同裝置和瀏覽器上的一致性。

主要特點：

- **響應式設計：** Bootstrap 採用移動優先的開發策略，確保網站在各種裝置上都有良好的顯示效果。
- **預設樣式：** 提供一套統一的樣式，涵蓋排版、按鈕、表單、導航等，讓開發者能快速建立一致的介面。
- **可擴充性：** 透過 CSS 變數和 Sass 變數，開發者可以輕鬆自訂 Bootstrap 的樣式，以符合專案需求。
- **豐富的元件：** 包含多種 UI 元件，如模態視窗、下拉選單、輪播等，提升開發效率。
- **社群支持：** 擁有龐大的開發者社群，提供豐富的資源、範例和插件，協助解決開發過程中的問題。

總而言之，Bootstrap 是一個強大且靈活的前端框架，能有效提升開發效率，並確保網站在各種裝置上的一致性。

Bootstrap 5 繁體中文文件

<https://bootstrap5.hexschool.com/docs/5.1/getting-started/introduction/>

安裝 bootstrap 套件

例如: 在 E:\WorkSpace\react-app\my-react-todolist> 下安裝

請根據您自己的 react 專案路徑進行安裝

```
npm install bootstrap
```

引入 bootstrap 資源

例如: 在 App5.jsx 中引入 bootstrap 資源

```
import 'bootstrap/dist/css/bootstrap.min.css'; // 引入 Bootstrap 樣式
```

Bootstrap 類別及其功能說明(React TodoList 專案中有使用的)：

類別名稱	功能描述
container	建立固定寬度且水平置中的容器，用於包裹主要內容。
mt-5	設定元素的上邊距 (margin-top) 為 Bootstrap 預設間距的 5 倍。
text-center	將文字內容置中對齊。
mb-4	設定元素的下邊距 (margin-bottom) 為 Bootstrap 預設間距的 4 倍。
input-group	將多個輸入元素組合在一起，形成一個輸入組合，通常用於將輸入框與按鈕結合。
mb-3	設定元素的下邊距 (margin-bottom) 為 Bootstrap 預設間距的 3 倍。
form-control	為 <code><input></code> 、 <code><textarea></code> 等表單元素提供預設樣式，使其外觀一致且美觀。
btn	為按鈕元素提供基本的按鈕樣式。
btn-primary	將按鈕樣式設為主要按鈕，通常使用主題色彩（預設為藍色）。
list-group	建立一個列表組，用於顯示一組相關的內容，如待辦事項清單。
list-group-item	列表組中的單個項目，提供預設的樣式和間距。
d-flex	將元素設為彈性盒模型 (flexbox)，方便進行彈性佈局。
justify-content-between	在彈性盒模型中，將子元素在主軸上兩端對齊，通常用於在左右兩側放置元素。

align-items-center	在彈性盒模型中，將子元素在交叉軸上置中對齊，通常用於垂直置中。
list-group-item-success	將列表項目的樣式設為成功狀態，通常使用綠色背景，表示任務已完成。
me-2	設定元素的右邊距 (margin-end) 為 Bootstrap 預設間距的 2 倍。
btn-danger	將按鈕樣式設為危險狀態，通常使用紅色背景，表示刪除或警告操作。
btn-sm	將按鈕設為小尺寸，適合在空間有限的地方使用。

這些 Bootstrap 類別的組合使用，使您的應用程式具有一致且美觀的外觀，並提升了開發效率。

App6.jsx 與後台 springboot 連接

使用 `useEffect()` 來連接後端 Rest API

Todo List Web API 文件

此文件提供了 Todo List Web API 的概述，包括管理待辦事項的端點。該 API 設計用於基本的 CRUD 操作。

基本 URL

<http://localhost:8080/todolist>

通用回應結構

所有回應都包裝在 `ApiResponse` 對象中：

- `status` (字串): 狀態訊息，例如 "success" 或 "error"。
- `message` (字串): 提供附加資訊的訊息。
- `data` (對象): 實際的回應數據（可以為 null）。

端點

操作	URL	方法	描述	JSON	回應 (JSON)
獲取所有待辦事項	/	GET	獲取所有待辦事項	無	<pre>{ "status": "success", "message": "查詢成功", "data": [{ "id": 1, "title": "Task A", "completed": false }] }</pre>

Java 企業框架實務班課程講-TodoList 範例

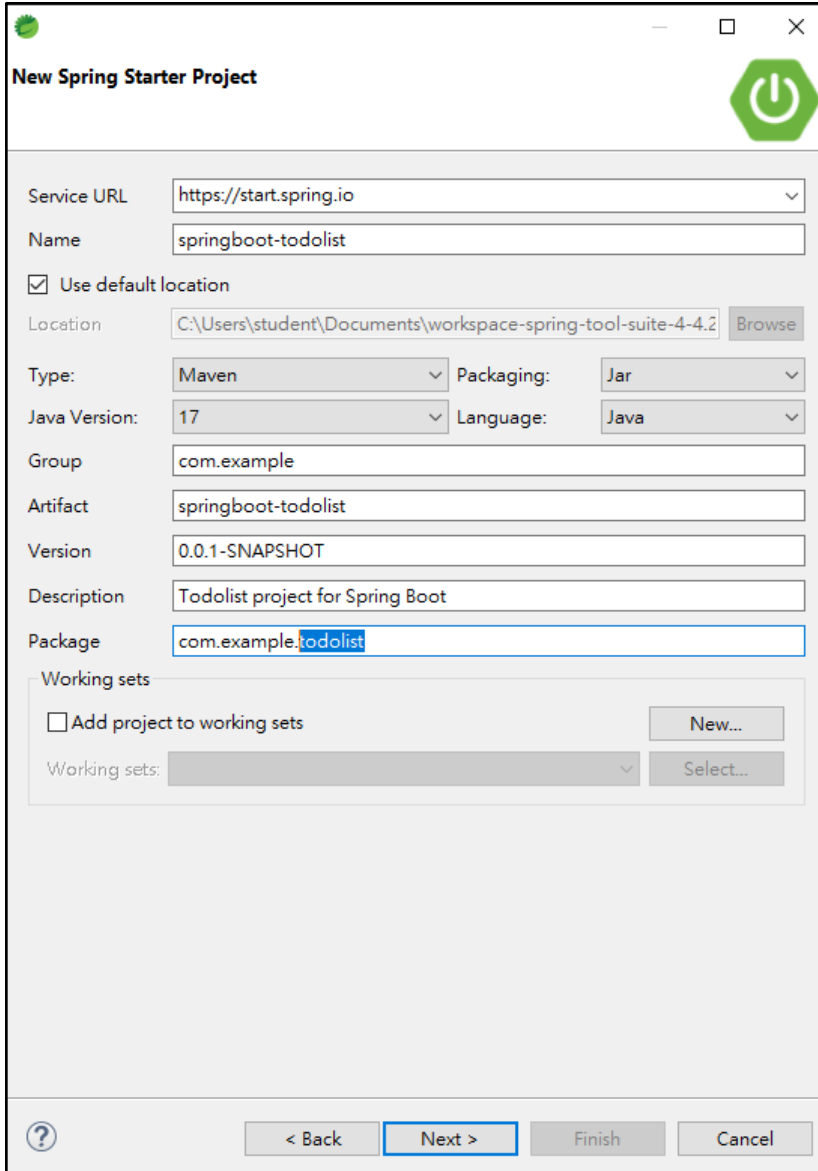
新增待辦事項	/	POST	新增一個待辦事項	{ "title": "New", "completed": false }	{ "status": "success", "message": "新增成功", "data": { "id": 2, "title": "New Todo", "completed": false } }
更新待辦事項	/id	PUT	根據 ID 更新待辦事項	{ "title": "Updated", "completed": true }	{ "status": "success", "message": "更新成功", "data": { "id": 2, "title": "Updated Todo", "completed": true } } 或 { "status": "error", "message": "更新失敗", "data": null }
刪除待辦事項	/id	DELETE	根據 ID 刪除待辦事項	無	{ "status": "success", "message": "刪除成功", "data": null } 或 { "status": "error", "message": "刪除失敗", "data": null }

CORS 配置

- 允許的來源: <http://localhost:5173>

該 API 允許來自託管於 <http://localhost:5173> 的前端應用程序的跨域請求。

Springboot 專案結構



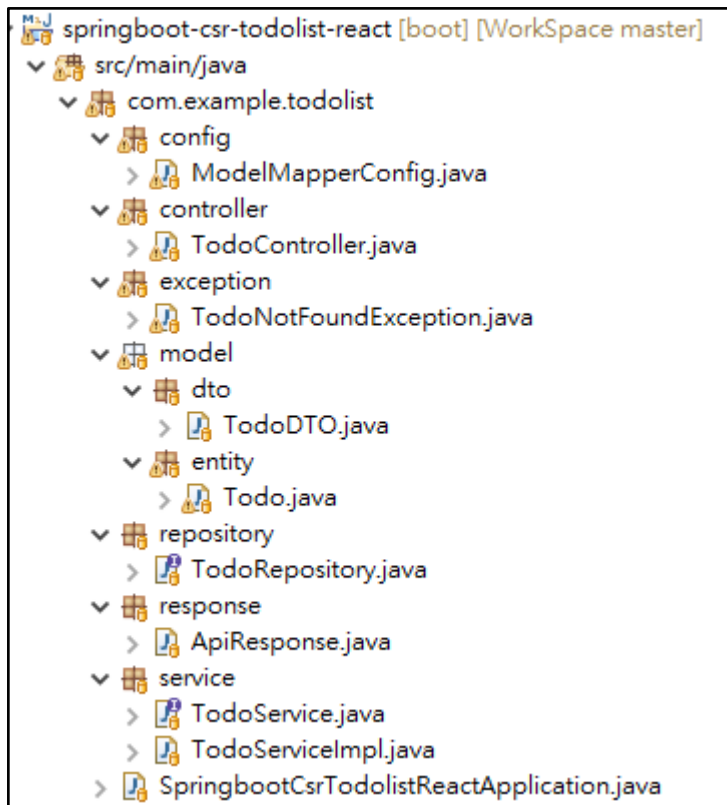
The image shows the 'New Spring Starter Project' dialog box in the Spring Tool Suite. The dialog is titled 'New Spring Starter Project' and features a green power button icon in the top right corner. The fields are as follows:

- Service URL: `https://start.spring.io`
- Name: `springboot-todolist`
- ☒ Use default location
- Location: `C:\Users\student\Documents\workspace-spring-tool-suite-4-4.2` (with a 'Browse' button)
- Type: `Maven` (dropdown)
- Packaging: `Jar` (dropdown)
- Java Version: `17` (dropdown)
- Language: `Java` (dropdown)
- Group: `com.example`
- Artifact: `springboot-todolist`
- Version: `0.0.1-SNAPSHOT`
- Description: `Todolist project for Spring Boot`
- Package: `com.example.springboot`

Below the fields, there is a 'Working sets' section with the following options:

- ☐ Add project to working sets (with a 'New...' button)
- Working sets: (dropdown menu) (with a 'Select...' button)

At the bottom, there are four buttons: '?', '< Back', 'Next >', 'Finish', and 'Cancel'. The 'Next >' button is highlighted with a blue border.



pom.xml

```
<dependencies>
```

```
    <!-- Model Mapper DTO 與 Entity 互轉 -->
```

```
    <dependency>
```

```
        <groupId>org.modelmapper</groupId>
```

```
        <artifactId>modelmapper</artifactId>
```

```
        <version>3.2.1</version>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
    </dependency>
```

```
    <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

SpringAI + Ollama



下載安裝與測試 Ollama

下載 Ollama

<https://ollama.com/download>

選擇模型

<https://ollama.com/search>

安裝模型

```
ollama pull gemma3:12b
```

測試模型

```
ollama run gemma3:12b
```

移除模型

```
ollama rm gemma3:12b
```

查看已安裝模型清單

```
ollama list
```

輸出範例

NAME	SIZE	MODIFIED
gemma3:12b	8.1 GB	2025-05-13
phi3:latest	4.2 GB	2025-05-12

Springboot + SpringAI 配置

設定 pom.xml

```
<dependencies>

...

<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-ollama-spring-boot-starter</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>1.0.0-M6</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

配置 application.properties

先確認有哪些模型？

指令 ollama list

將 deepseek-r1:14b 配置到 application.properties

```
# 配置 model  
spring.ai.ollama.chat.model=gemma3:12b
```

專案配置

AI Agent

(課後補充用)



AI Agent 與 MCP：推動智慧自動化的新世代關鍵技術

AI Agent 一次看懂！AI 大咖口中的未來趨勢

<https://www.youtube.com/watch?v=hH2HkAqOPxE&t=65s>

MCP 當你的「AI 翻譯」，實現黃仁勳口中的 AI Agent 全靠它？

<https://www.youtube.com/watch?v=ssc0Co3S6go&t=39s>

何謂 AI Agent ？

AI Agent (人工智慧代理人) 是一種能夠自主執行任務的軟體系統，具備理解、規劃、決策與行動能力。它能根據使用者的高階目標，主動分析環境、調用外部工具或數據，並分步完成複雜任務，而不僅僅是被動回應指令。

AI Agent 的核心特色包括：

- 自主性：能根據目標自行決定行動步驟，主動規劃並執行多階段任務。
- 智慧決策：結合大型語言模型 (LLM)、推理與記憶模組，理解複雜需求並做出最佳決策。
- 多工具整合：可調用 API、資料庫、外部應用等多元資源，完成跨平台、跨系統工作。
- 持續學習與適應：能根據過去互動經驗優化未來行為，提升個人化與效率。

AI Agent 與 MCP 的關係

MCP (Model Context Protocol，模型上下文協定) 是一套開放標準，專為解決 AI Agent 與外部世界整合的難題而設計。MCP 提供統一且標準化的介面，讓 AI Agent 能安全、即時地存取外部資料、調用工具，甚至直接執行操作。

兩者的關係可簡述如下：

- MCP 是 AI Agent 的「通用連接器」
傳統上，AI 雖然能理解指令，卻難以直接操作外部系統或取得即時資料。MCP 則像「AI 的 USB-C」，讓不同 AI Agent 能輕鬆連接各種應用、資料庫、雲端服務與本地資源，突破資訊孤島與整合瓶頸。
- MCP 擴展 AI Agent 的行動範圍
有了 MCP，AI Agent 不再侷限於單一平台或靜態知識，而能主動存取最新資訊、執行跨系統任務，真正成為「能做事」的智慧助理。
- MCP 標準化互動流程，降低開發複雜度
開發者無需為每個外部工具重複設計專屬串接，只要支援 MCP 協定，AI Agent 就能自動發現、調用並安全互動，大幅提升開發效率與系統彈性。

簡單來說：

LLM (大型語言模型) 本來只會「說話」和「寫字」，但有了 MCP 這個橋樑，LLM 說出的特定格式文字，會被 MCP 轉換成指令，**去控制各種軟體和硬體**。

這樣一來，LLM 不只會聊天，還能幫你查資料、下指令、甚至遠端操作設備，變成真正會「**動手做事**」的智慧助理！

總結來說，AI Agent 是智慧自動化的主角，而 MCP 則是讓這些 AI Agent 能真正「動手做事」、與世界無縫連接的關鍵基礎設施。兩者結合，正推動各行各業邁向更高層次的智能化與自動化。

為何 AI Agent 會火紅？

- 自動化與效率大幅提升：AI Agent 不再只是被動回應，而是能主動理解需求、規劃並執行多步驟任務，從搜尋、比價到下單、追蹤全自動完成，大幅減少人力與時間成本。
- 個人化與智慧決策：AI Agent 能即時分析用戶偏好與情境，提供高度個人化的建議與服務，甚至主動預測並滿足需求，這是傳統工具難以比擬的。
- 跨平台與多工具整合：新一代 AI Agent 可以直接操作各種應用、API 和數據源，無縫串接不同服務，帶來前所未有的數位體驗。

- 用戶行為轉變：隨著 AI Agent 能直接對話、執行任務，用戶將不再需要手動瀏覽、點擊多個網頁，而是直接用自然語言下指令，AI 便能完成所有操作。

傳統瀏覽器會被淘汰嗎？

- 趨勢上，AI Agent 的崛起確實可能讓傳統瀏覽器的「主動搜尋、被動點擊」模式逐漸式微。未來的資訊獲取與網路互動，將更多依賴 AI Agent 以對話、任務導向的方式完成。
- 不過，專家普遍認為，短期內 AI Agent 會與瀏覽器深度融合，瀏覽器將內建 AI 助手、智能搜尋、任務自動化等功能，提升效率與個人化體驗，而非完全消失。
- 長遠來看，若 AI Agent 能完全取代現有瀏覽、搜尋、交易等流程，傳統瀏覽器的角色就會被大幅弱化，甚至被嵌入式、無界面、全自動的 AI 互動模式取代。

小結

AI Agent 之所以火紅，正是因為它代表了人機互動邁向自動化、個人化、無縫整合的新時代。隨著大眾對 AI 工具的依賴加深，傳統瀏覽器將逐步被重新定義、融合甚至邊緣化，未來的數位生活將以 AI Agent 為中心，徹底改變我們與網路世界的互動方式。

AI Agent 之所以成為熱門趨勢，確實與未來大眾將普遍使用 AI 工具密切相關，並且這種轉變有可能讓傳統瀏覽器逐步被邊緣化，但並非單純「淘汰」而是「**重塑**」與「**融合**」。

AI Agent 企業內部導入現況

- 主流應用場景：AI Agent 在企業內部被廣泛用於自動化員工支援、HR、IT 支援、客服、軟體開發、銷售等多種業務流程，幫助企業提升效率、降低成本並改善用戶體驗。
- 採用比例高：2025 年，超過 70% 的 **AI 導入專案聚焦於「可執行任務」的 AI Agent**，而不僅僅是聊天機器人。85% 的企業預計會在業務流程中採用 AI Agent。
- 企業推動力強：AI Agent 能帶來高達 50% 的效率提升，這成為企業快速推動內部導入的關鍵動力。
- 產業範圍廣：科技、金融、保險、製造等產業是目前 AI Agent 應用最積極的領域，許多 Fortune 500 企業已在內部流程導入 AI Agent。

消費端應用現況

- 雖然消費者也在日常生活中使用 AI 助手（如 Google Assistant、Siri、Alexa），但這些多屬於語音助手或簡單任務自動化，與企業內部能整合多系統、執行複雜任務的 AI Agent 相比，功能與影響力仍有限。

發展趨勢

- 企業內部先行，逐步外溢：目前 AI Agent 以企業內部流程自動化為主，未來隨著技術成熟與標準化（如 MCP 協議），將逐步擴展到消費端與更多產業應用。
- 安全與合規為企業首要考量：企業偏好將 AI Agent 部署於自有雲端或內部系統，以確保資料安全與合規。

結論：

現階段 AI Agent 的應用確實以企業內部為主，驅動企業數位轉型與效率提升。隨著技術發展與標準普及，未來 AI Agent 將進一步滲透到消費端與更多生活場景

AI Agent 與 MCP 技術革新網路購物體驗

隨著人工智慧 (AI) 技術的快速發展，網路購物的模式也在持續演變。從最初的傳統購物平台，到引入 AI 智能推薦，再到結合先進的 Model Context Protocol (MCP) 技術，使 AI 不僅能提供建議，更能實際執行購物流程，帶來前所未有的便利與效率。本文將透過一個具備 AI Agent 的網路購物情境，深入解析傳統網路購物、有 AI 的網路購物與有 MCP 的網路購物之間的差異，並探討 MCP 技術帶來的優勢、商機與未來發展潛力。

傳統網路購物

- 使用者自行瀏覽商品頁面、比較價格、加入購物車，並完成結帳流程。
- 主要依賴使用者主動搜尋與決策，缺乏智能輔助。
- 客服多為人工或簡單的 FAQ 系統，無法提供即時且個人化的服務。

有 AI 的網路購物

- AI 可以提供商品推薦、搜尋優化、聊天機器人客服等功能。
- AI 多半是「建議者」角色，例如告訴你哪些商品可能喜歡，或回答常見問題。
- AI 雖然能理解用戶需求，但通常無法直接操作系統或執行複雜任務，仍需用戶手動完成訂單、付款等步驟。

有 MCP (Model Context Protocol) 的網路購物

- MCP 讓 AI 從「只能說」的建議者，進化為真正的「AI Agent」，能直接執行任務。
- AI Agent 可透過 MCP 連接購物平台的後端系統，自動幫用戶完成商品搜尋、比較、加入購物車、下單、付款甚至追蹤訂單。
- 例如，使用者只需告訴 AI「幫我買這款手機，預算不超過 1 萬元，並且選擇最快配送」，AI Agent 就能直接操作平台完成訂單，並在訂單狀態更新時主動通知用戶。

- AI Agent 也能主動管理會員積分、優惠券使用，甚至自動處理退換貨流程，提升購物體驗的便利性與效率。
-

MCP 的優勢與商機

優勢

- 執行力強：AI 不只是提供建議，而是能直接操作多個系統，完成複雜任務，解放使用者雙手。
- 跨系統整合能力：MCP 讓 AI 能連接不同平台和服務（如支付系統、物流、客服系統），實現端到端自動化。
- 去中心化與透明性：結合區塊鏈技術，MCP 協議能保障數據安全、交易透明，並促進多方合作及資源共享。
- 個人化與主動服務：AI Agent 可根據用戶歷史和偏好，主動提供個性化推薦和服務，甚至預測需求。

商機與未來性

- MCP 是 AI 與區塊鏈融合的創新協議，將推動 AI 服務的去中心化和資產化，創造新的經濟模式。
- 在電子商務、金融、醫療、製造等多個行業均有廣泛應用潛力，尤其能提升自動化和智能化水平。
- MCP 促進產業鏈整合，打破資料孤島，推動數據和計算資源共享，為企業帶來成本降低和效率提升。
- 隨著 MCP 生態系統擴大，基於 MCP 的 AI 服務和資產將成為數位經濟的重要組成部分，吸引大量資本投入，帶來長期成長機會。
- 未來 MCP 將使 AI Agent 不僅是工具，更是獨立的智能經濟體，能自主管理資源、創造價值，重新定義工作與生活。

總結

傳統網路購物依賴用戶手動操作，有 AI 的網路購物則提供智能建議和客服，但仍需用戶執行操作；而結合 MCP 的網路購物，AI Agent 不僅能理解需求，更能直接執行購物全流程，實現真正的智能自動化。MCP 技術的出現，將帶來電子商務乃至整個數位經濟的革命性變革，具有廣闊的市場前景和商業價

項目	傳統網路購物	有 AI 的網路購物	有 MCP 的網路購物 (AI Agent)
使用者互動	使用者主動搜尋、選擇、下單	AI 提供推薦、客服，使用者仍需操作	AI Agent 可理解需求並直接執行購物流程
功能範圍	基本瀏覽、比較、購買	智能推薦、聊天客服	自動搜尋、下單、付款、追蹤訂單，甚至退換貨處理
整合能力	單一平台操作	AI 與平台有限整合	MCP 協議支援跨系統、跨服務整合，動態工具發現與即時互動
自動化程度	低	中等	高，AI Agent 可主動完成複雜任務
即時資料接取	無	部分依賴平台數據	支援即時資料接取，反映最新庫存、價格等資訊

個人化服務	靜態依賴使用者設定	基於歷史數據與偏好推薦	高度個人化，且在保護隱私的前提下動態存取必要資訊
使用者體驗	需較多手動操作	較為便捷，但仍需用戶參與	流暢無縫，AI Agent 如同個人助理，解放雙手
安全與隱私	傳統安全機制	依平台與 AI 服務而異	MCP 提供標準化安全控制，跨工具一致保護用戶資料
擴充性與維護	需手動新增功能與維護	依平台與 AI 服務更新	MCP 支援即插即用，標準化接口易於擴充與維護

AI Agent 與 MCP 技術革新網路股票下單體驗

隨著金融科技 (FinTech) 與人工智慧 (AI) 的發展，股票下單方式正經歷重大轉型。從最初的傳統網路下單，到引入 AI 智能輔助，再到結合 Model Context Protocol (MCP) 協議的 AI Agent，投資人下單流程日益自動化與個人化。本文將以網路股票下單為例，說明傳統、AI 輔助與 MCP 技術下單的主要差異，並以比較表格呈現三者 在功能、效率與體驗上的不同。

差異說明

傳統網路股票下單

- 投資人需自行登入券商平台，查詢行情、輸入股票代碼、數量與價格，手動送出委託單。
- 需自行追蹤成交狀況與管理持股，遇到問題多半依賴人工客服。
- 下單流程完全由投資人主導，缺乏智能輔助與自動化。

AI 網路股票下單

- AI 可協助分析市場數據、預測趨勢、提供下單建議，或透過聊天機器人回答投資相關問題。
- 投資人可根據 AI 建議調整策略，但實際下單仍需手動操作。
- AI 主要扮演「建議者」角色，提升決策效率，但無法自動執行下單或跨平台整合。

MCP 網路股票下單 (AI Agent)

- AI Agent 透過 MCP 協議，能直接連接多家券商與金融服務，根據投資人需求自動下單、調整委託、追蹤持股與風險管理。
- 投資人只需下達高階指令 (如「根據我的風險偏好，幫我自動配置台灣 50 成分股」)，AI Agent 即可自動執行並主動回報。
- 支援跨平台、跨資產整合，並可自動處理停損、停利、再平衡等複雜任務，實現真正的智能投資自動化。

比較表格：傳統、AI 與 MCP 網路股票下單差異

項目	傳統網路股票下單	AI 網路股票下單	MCP 網路股票下單 (AI Agent)
下單方式	投資人手動輸入、送出委託	AI 提供分析與建議，投資人手動下單	AI Agent 理解需求後自動執行下單與管理

Java 企業框架實務班課程講-AI Agent(課後補充用)

市場分析	需自行查詢、分析	AI 協助分析、預測、推薦	AI Agent 可即時分析、決策並自動執行
自動化程度	幾乎無自動化	部分自動化 (如預警、推薦)	高度自動化，從分析到下單全流程自動完成
跨平台整合	僅限單一券商	依券商與 AI 能力	支援多券商、多資產整合，動態工具發現與即時互動
風險管理	需自行設置停損、停利	AI 可提醒風險，但需手動調整	AI Agent 可自動調整停損、停利、再平衡
個人化服務	靜態依賴用戶設定	根據歷史數據與偏好提供建議	高度個人化，主動調整投資組合與策略
客服支援	人工客服為主	AI 聊天機器人輔助	AI Agent 可主動偵測異常並即時回應
安全與隱私	傳統安全機制	依券商與 AI 服務而異	MCP 標準化安全控制，跨券商一致保護用戶資料

擴充性與 維護	需手動新增功能與 維護	依券商與 AI 服務更新	MCP 標準接口，易於擴充與多平台 維護
------------	----------------	--------------	-------------------------

此表格清楚展示 MCP 技術如何讓 AI Agent 具備跨平台整合、自動化執行與高度個人化等優勢，徹底改變傳統與 AI 輔助下單的投資體驗。

AI 與 MCP 技術革新 IT 課程購買與補習產業挑戰

隨著數位學習與人工智慧的發展，IT 課程的購買與學習方式正快速演變。過去多數人會親自前往傳統補習班門市選課，近年則興起 AI 智能推薦與線上課程平台。未來，若結合 MCP (Model Context Protocol) 協議，AI Agent 不僅能理解需求，更可自動整合多平台、個人化規劃學習路徑，帶來全新體驗。本文將比較三種模式下的 IT 課程購買差異，並分析補習業者若未跟進 MCP 技術可能面臨的隱憂。

差異說明

傳統實體補習班購買 IT 課程

- 學員需親自到傳統補習班門市諮詢、選課、報名與繳費。
- 需與現場顧問溝通需求，課程安排與時間彈性有限，常受地點、時段與開班人數影響。
- 課程內容、就業輔導與服務多依賴人工，學員需主動管理學習進度與資源。

AI 網路購買 IT 課程

- 學員可透過線上平台或 AI 智能推薦系統，依興趣、職涯目標自動獲得課程建議。
- AI 可分析學習歷程與市場趨勢，提供個人化課程組合，並協助解答常見問題。
- 實際選課、付款與進度管理仍需學員自行操作，AI 主要扮演「建議者」角色。

MCP 購買 IT 課程 (AI Agent)

- AI Agent 透過 MCP 協議，能跨平台整合多家補習班與線上課程資源，自動為學員規劃最適合的學習路徑。

Java 企業框架實務班課程講-AI Agent(課後補充用)

- 學員只需簡單描述需求（如「我要在半年內學會 Python 並取得證照」），AI Agent 即可自動搜尋、比較、報名、付款，並主動追蹤學習進度、提醒課程、協助申請證照與就業媒合。
- 支援即時調整學習計畫，根據學員表現與市場變化自動優化課程選擇，實現真正的個人化與自動化學習體驗。

比較表格：三種 IT 課程購買模式

項目	傳統實體補習班購買	AI 網路購買 IT 課程	MCP 購買 IT 課程 (AI Agent)
購買流程	親自到班諮詢、選課、繳費	線上平台 AI 推薦，學員自行操作	AI Agent 理解需求後自動跨平台規劃、報名、付款
課程選擇	受限於地點、時段、開班人數	多元平台，AI 提供建議	跨平台整合，動態組合最適課程
個人化程度	需與顧問溝通，彈性有限	AI 根據偏好推薦，仍需手動挑選	高度個人化，全自動調整學習路徑
學習管理	學員自行追蹤進度、補課	AI 可提醒進度，管理仍靠學員	AI Agent 主動追蹤、調整、協助證照與就業

客服與支援	現場人工諮詢	AI 聊天機器人輔助	AI Agent 主動偵測問題並即時協助
擴充性	受限於單一補習班	依平台內容為主	跨平台、跨領域即插即用

補習業者若不實現 MCP 的挑戰與隱憂

- **競爭力下降**：消費者習慣 AI Agent 自動化、個人化服務後，傳統補習班若無法支援 MCP，將難以吸引新世代學員，流失市場份額 35。
- **數據孤島問題**：未整合 MCP，課程、學員與資源無法跨平台流通，無法有效利用大數據優化教學與招生 3。
- **營運效率落後**：缺乏自動化與智能化流程，招生、服務與課程管理成本高，難以應對多元學習需求 3。
- **品牌與服務創新受限**：無法提供動態調整、即時回應與全方位學習路徑，品牌形象與用戶體驗落後於數位時代 3。

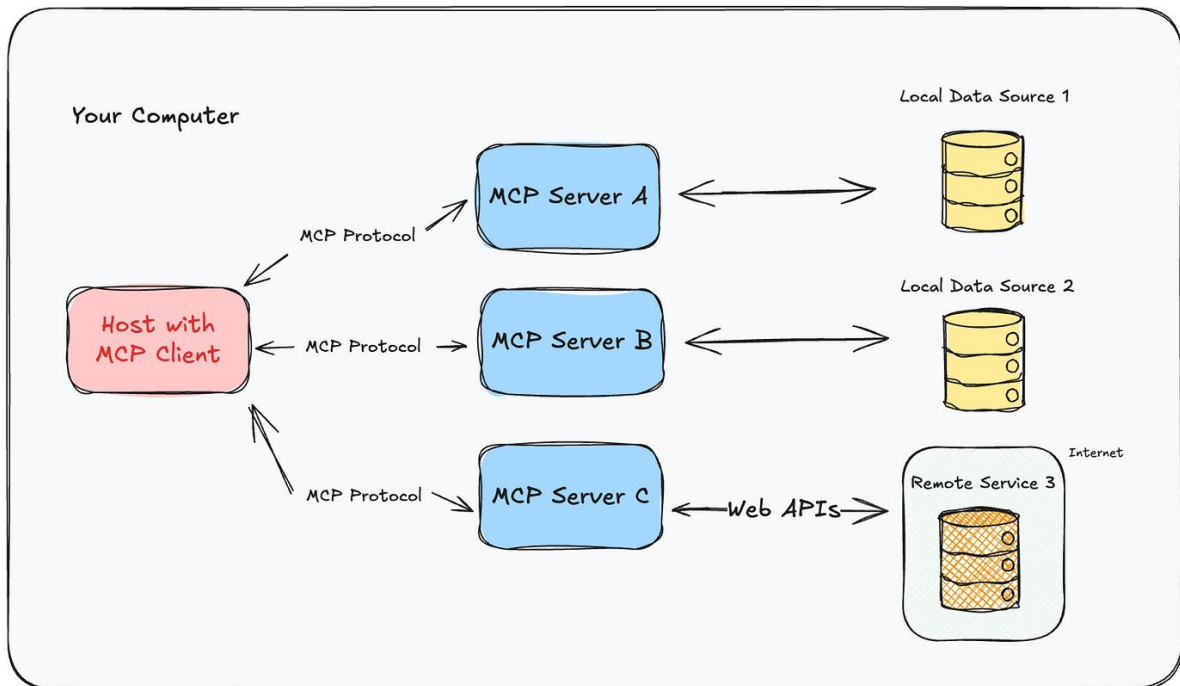
「數位化轉型還需要補習班行業主動應對數位挑戰。隨著教育需求和消費行為的變化，補習班需要更高效地進行管理和溝通。整合數位平台來管理課程、學生資訊和教學資源，或採用創新方法吸引學生和家長，可以大大提高效果。」 3

小結

MCP 技術將補習產業推向高度自動化、個人化與跨平台整合的新時代。補習業者若未及時轉型，將面臨市場競爭力下降、營運效率落後與品牌創新受限等重大挑戰。

MCP 調用流程





Spring AI MCP 架構與官方設計理念：

- MCP Client 負責解析 prompt，將其拆解為 tool call (如 buyStock、sellStock、viewPortfolio)，然後根據工具所屬，分別呼叫不同的 MCP Server。
- MCP Server 1 提供 buyStock、sellStock 這兩個工具 (Tool)。
- MCP Server 2 提供 viewPortfolio 這個工具 (Tool)。
- MCP Client 根據工具註冊情況，自動將 call 對應到正確的 server

未來企業的核心價值



未來企業的核心價值

根據最新產業觀點與專家分析，未來企業系統的核心價值確實將逐步轉向 AI Agent。

以下是重點說明：

- AI Agent 不只是輔助工具，而是企業流程與決策的主動參與者，能自動協調多系統、優化工作流程、做出即時決策，並持續學習與適應市場變化。
- AI Agent 具備自主規劃、執行、協作與跨平台整合能力，能大幅提升企業效率、創新與競爭力，並釋放人力專注於更高價值的創造。
- 越來越多企業將 AI Agent 視為未來數位轉型和智能自動化的關鍵基礎設施，預期到 2028 年，三分之一的企業軟體都將內建 agentic AI(代理式人工智慧)
- <https://www.seekr.com/blog/understanding-ai-agents-the-next-step-in-enterprise-transformation/>

預測

 巨匠教育

移動平均法:

移動平均法的基本概念是計算近期的平均數據，並用此來預測下一期的數據。

例如，我們可以計算最後三個數字的平均值來預測下一個數字。

當日 + 前一日 + 前二日

$$\text{三日移動平均} = \frac{\text{-----}}{3}$$

案例一:

小華在學校經營一個飲料攤位。他記錄了過去四天的飲料銷售量（瓶）如下：

星期一：12 瓶

星期二：14 瓶

星期三：15 瓶

星期四：13 瓶

小華想使用三日移動平均法來估計星期五的銷售量。請根據上述數據，幫助小華預測星期五可能的銷售量。

星期三的三日移動平均 = $(12 + 14 + 15) / 3 = 13.67$ (預估星期四) 實際值 13

星期四的三日移動平均 = $(14 + 15 + 13) / 3 = 14$ (預估星期五)

案例二：

某公司最近五天的股價為：\$50, \$52, \$50.5, \$51.5, \$52。利用三日移動平均法，請預測該公司第六天的股價。

$(\$50.5, \$51.5, \$52) / 3 = 51.33$ (預估第六天的股價)

指數平滑 (Exponential Smoothing)

- * 指數平滑法是一種時序預測技術，它基於對過去數據的加權平均進行預測。
- * 權重會隨著數據點距離預測點的遠近而遞減。
- * 這個方法特別適合於當數據在短期內有一定的規律性，但長期趨勢不明確的情況。
- * 應用: 主要用於時間序列數據預測。
- * 數學公式：
- * $F_{t+1} = \alpha Y_t + (1 - \alpha)F_t$
- * F_{t+1} 是下一期的預測值
- * Y_t 是當期的實際值
- * F_t 是當期的預測值
- * α 是平滑係數 (通常在 0 到 1 之間)
- * 案例:
- * 小華最近開始關注某公司的股價。在連續三天中，這家公司的股價分別是：\$10、\$12 和 \$15。
- * 小華想使用指數平滑法預測這家公司的股價在第四天可能是多少(如果平滑係數 α 為 0.3)。

$$Y_1 = 10$$

$$Y_2 = 12$$

$$Y_3 = 15$$

$$\alpha = 0.3$$

第一天的預測(初始預測)

$$F_1 = Y_1 = 10$$

根據第一天的數據預測第二天 ($t=1$)

$$\begin{aligned} F_2 &= \alpha Y_1 + (1 - \alpha)F_1 \\ &= 0.3 \times 10 + 0.7 \times 10 \\ &= 10 \end{aligned}$$

根據第二天的數據預測第三天 ($t=2$)

$$\begin{aligned} F3 &= \alpha Y2 + (1 - \alpha)F2 \\ &= 0.3 * 12 + 0.7 * 10 \\ &= 10.6 \end{aligned}$$

根據第三天的數據預測第四天 (t=3)

$$\begin{aligned} F4 &= \alpha Y3 + (1 - \alpha)F3 \\ &= 0.3 * 15 + 0.7 * 10.6 \\ &= 11.92 \end{aligned}$$

所以，使用指數平滑法，小華可以預測第四天的股價將是 \$11.92。

線性回歸 (Linear Regression)

* 應用: 通常用於預測連續的數值。

* 線性回歸的目的是找到一條直線 $y = mx + b$ 來最好地適合資料點集。

* 數學公式:

$$y = mx + b$$

* 其中，

* m 是斜率，它描述了 y 隨 x 變化的速率或方向

* b 是截距，是當 $x = 0$ 時 y 的值。換句話說，它表示了當自變量 x 為 0 時，因變量 y 的基線或起始值。

*

* 在線性回歸中，斜率 m 和截距 b 的公式如下

* 斜率 m :

$$\frac{n(\sum xy) - (\sum x)(\sum y)}{n(\sum x^2) - (\sum x)^2}$$

* 截距 b :

$$\frac{(\sum y) - m(\sum x)}{n}$$

n 是資料點的數量。

$\sum x$ 是所有 x 值的總和。

$\sum y$ 是所有 y 值的總和。

$\sum xy$ 是所有 x 和 y 值乘積的總和。

$\sum x^2$ 是所有 x 值平方的總和。

問題 1:

小華在連續三天的數學考試中，獲得了以下分數。他想知道，如果這個趨勢繼續下去，他在第四天的考試中可能會得到多少分？

資料如下：

考試天數 (x) 得分 (y)

1 80

2 82

3 84

請使用線性回歸的方法預測小華在第四天考試的分數。

$$n = 3$$

$$\sum x = 1 + 2 + 3 = 6$$

$$\sum y = 80 + 82 + 84 = 246$$

$$\sum xy = 80*1 + 82*2 + 84*3 = 496$$

$$\sum x^2 = 1^2 + 2^2 + 3^2 = 14$$

* 斜率 m:

$$\frac{3(496) - (6)(246)}{3(14) - (6)^2} = 2$$

* 截距 b:

$$\frac{(246) - 2(6)}{3} = 78$$

$$m = 2, b = 78$$

* 線性回歸數學公式:

* $y = mx + b$ (x=4 表示第四天, y 則是第四天的分數)

$$= 2*4 + 78$$

$$= 86$$

ARIMA-自迴歸整合移動平均模型

AR: 自迴歸, I: 差分, MA: 移動平均

ARIMA 模型 (全稱為「自迴歸整合移動平均模型」, Autoregressive Integrated Moving Average Model) 是預測時間序列數據的一種常用統計模型。它結合了自迴歸 (AR)、差分整合 (I) 和移動平均 (MA) 三種方法, 以處理各種不同類型的時間序列數據, 尤其是那些具有趨勢和季節性的數據。

ARIMA 模型的三個主要組件:

* 自迴歸 (AR) :

* 自迴歸部分反映了當前數據點與其過去值之間的關係。它假設當前觀察值可以用過去若干期的觀察值的線性組合來進行預測。

模型中的 p 代表自迴歸項的數量。

比喻: 你今天喝幾杯水, 和你前幾天喝幾杯有關。

例如: 你發現自己習慣上, 前一天喝得多, 今天也會喝得多。這就是「自回歸」: 用過去 p 天的喝水量來預測今天。

* 差分整合 (I) :

* 差分是為了使非穩定時間序列變得穩定, 通過計算連續觀察值之間的差異來去除序列的趨勢或季節性。 d 代表進行差分的次數。

比喻: 你發現喝水量有明顯的上升或下降趨勢, 必須先扣掉這種趨勢, 才能看出真正的規律。

例如: 最近天氣越來越熱, 你每天都比前一天多喝一杯。這時候要用「差分」: 把每天和前一天的差 (今天喝幾杯 - 昨天喝幾杯) 當作新的數列,

消除趨勢, 讓資料變得穩定。

* 移動平均 (MA) :

* 移動平均部分關注了時間序列與過去預測誤差之間的關係。這一部分的目的是捕捉時間序列中的隨機波動。模型中的 q 代表移動平均項的數量。

比喻: 你今天喝水的杯數, 除了和過去的習慣有關, 也會受到「昨天預測錯多少」的影響。

例如：你昨天本來預計會喝 6 杯，結果實際喝了 8 杯，今天你可能會修正預測，喝得比預期多。這就是「移動平均」：

用過去 q 天預測誤差來修正今天的預測。

* ARIMA 模型的特點：

- * 1. 能夠模擬和預測具有時間依賴性的數據。
- * 2. 適用於非季節性和季節性的時間序列預測。
- * 3. 通過差分轉換，可以處理非穩定數據。
- * 4. 需要對時間序列進行仔細的分析和參數選擇。

*

* 應用：

* ARIMA 模型廣泛應用於經濟數據、自然現象、股票市場等時間序列的預測中。

* 在股票市場中，ARIMA 模型可以用來預測未來某支股票的價格。讓我們通過一個例子來簡單介紹自迴歸 (AR)、差分 (I) 和移動平均 (MA) 這三個概念：

* 想象您正在觀察一家公司的股票價格，希望能夠預測明天的股價。

*

* 自迴歸 (AR)：

* 自迴歸部分的想法是，今天的股價與過去幾天的股價有關。

* 如果我們選擇了 $p=3$ 的自迴歸階數，那麼今天的股價可以看作是過去三天股價的一種加權組合。

* 這種加權反映了過去股價對當前股價的影響力度。舉個例子，如果過去幾天股價呈現上升趨勢，我們可能會預測今天的股價會繼續上升。

* 差分 (I)：

* 差分是為了讓非穩定的時間序列數據變得穩定。

* 在股市中，股價往往會受到很多因素的影響，例如市場情緒、公司新聞或經濟事件，這些都可能導致股價顯示出某種趨勢或季節性模式。

* 通過差分 (也就是連續計算股價的日漲跌幅度)，我們可以消除這些趨勢，讓模型專注於股價的波動本身，而不是其絕對值。

* 移動平均 (MA)：

- * 移動平均部分則是關注那些無法被模型通過趨勢解釋的隨機變化，也就是市場的“噪音”。
- * 例如，某家公司因為意外事件而導致的股價短期內的異常波動。
- * 通過加入 $q=2$ 的移動平均階數，模型會考慮到過去兩天的隨機震盪，並用這些資訊來幫助平滑預測。

* 選擇 ARIMA 模型的參數 (p,d,q) 時， p 代表模型中自迴歸項的數量， d 代表進行差分的次數，以使時間序列數據穩定， q 代表模型中移動平均項的數量。

* 在實際應用中，這些參數需要通過觀察數據本身的特性或進行統計測試來確定。

* 在股市分析的實際應用中，您需要試驗不同的參數組合，並使用如赤池資訊量準則 (AIC) 等標準來評估模型的好壞，以選擇最合適的 ARIMA 模型來進行預測。

* 這個過程稱為模型的“訓練”，一旦模型被“訓練”好，就可以用來預測未來的股價走勢。

*

*

* `ArimaOrder.order(1, 1, 1)` :

* $p=1$: 這表示我們假設股價與其前一天的價格存在線性關係。今天的股價部分由昨天的股價加上某個固定的加權值決定。

* $d=1$: 這表示我們對數據進行一次差分，以消除可能存在的趨勢性，使序列變得穩定。

* 在股價的例子中，這通常是將每一天的股價與前一天的股價相減，得到股價變動的數據，即日收益率。

* $q=1$: 這表示模型包含一個移動平均項，意味著今天的誤差 (即實際價格與模型預測價格之間的差異) 受到前一天誤差的影響。

*

* 總而言之，選擇不同的 d 值反映了對數據穩定性的不同假設。

* $d=1$ 通常用於處理有趨勢的數據，而 $d=2$ 用於處理更複雜的非穩定時間序列。

* 選擇 p 和 q 的值則是基於對數據自相關性質的理解，以及嘗試擬合數據中觀察到的模式。

* 在實際應用中，往往需要嘗試多種模型配置，並使用統計測試來決定

*

*

* `ArimaOrder.order(2, 2, 3)` :

* $p=2$: 這表示當前股價不僅與前一天的股價有關，還與前兩天的股價相關。換句話說，股價的今天值被認為是前兩天股價的加權函數。

* 這可能表明市場的記憶效應較長，投資者在做出買賣決策時會考慮到前兩天的價格信息。

* $d=2$: 這表示對原始股價序列進行了兩次差分，以期使序列達到平穩狀態。

* 在股價的例子中，一次差分通常是計算日收益率，而二次差分可能是因為數據中存在加速上升或下降的趨勢，或者存在更為複雜的非線性趨勢。

* 二次差分有助於移除這些複雜趨勢，使模型能夠更好地捕捉價格波動中的模式。

* $q=3$: 三個移動平均項意味著模型會考慮前三天的隨機震盪或噪音對今天股價的影響。

* 這反映出股價不僅受到過去幾天趨勢的影響，也受到近期市場隨機波動的影響。

* 這可以是由於市場的快速反應，如對於突發新聞或事件的短期反應。

*

* 總結來說，`ArimaOrder.order(2, 2, 3)` 的設置表明你認為股價的當前值與前兩天的值有較強的相關性，

* 股價序列包含複雜的趨勢需要通過二次差分來消除，並且近期的隨機誤差或震盪在短期內對股價有較大的影響。

* 這樣的模型可能適用於那些波動較大，趨勢變化較為複雜的股票價格序列。然而，這樣的模型需要謹慎選擇，以避免過度擬合和失去解釋力。

* 通常，對於實際的股價預測，我們會根據數據的特性、時間序列診斷以及模型選擇準則來確定合適的模型階數。

Smile 機器學習庫的股價預測器

Smile (Statistical Machine Intelligence and Learning Engine) 是一個用 Java 編寫的全面的機器學習函式庫，它提供了多種機器學習演算法和資料處理的方法。Smile 的設計目標是提供一個可以輕鬆使用的機器學習工具，同時保持高效能和靈活性。它包含的演算法涵蓋了分類、迴歸、聚類、關聯規則和維度約簡等多個領域。

* Smile 的特點包括：

- * 1. 多樣的演算法支援：Smile 提供了廣泛的機器學習演算法，
- * 2. 包括支援向量機 (SVM)、隨機森林、梯度提升樹 (GBT)、k-最近鄰 (k-NN)、邏輯迴歸、人工神經網路等。
- * 3. 易用性：Smile 旨在簡化機器學習的應用，它提供了清晰的 API 和豐富的文檔，使得開發人員可以快速理解和使用。
- * 4. 效能：Smile 在設計上著重效率和效能，可以處理大規模的資料集。
- * 5. 資料處理：Smile 不僅提供機器學習演算法，還包含了資料預處理、特徵抽取和資料轉換的工具。
- * 6. 視覺化：Smile 還包含了一些資料視覺化工具，用於繪製決策邊界、資料點、統計圖表等。
- * 7. 跨平台：作為一個 Java 庫，Smile 可以在任何支援 Java 的平台上運行，包括 Windows、Linux 和 macOS。

*

- * DataFrame 用於存儲和操作結構化數據。
- * DoubleVector 用於創建包含數字數據的列。
- * RandomForest.fit 方法用於訓練隨機森林模型。
- * Tuple 代表數據集中的一行數據，在這裡是用來做預測的最後一個觀察值。

*

隨機森林算法是一種集成學習方法，用於迴歸和分類問題。它建立於決策樹算法之上，通過結合多棵決策樹的預測結果來提高整體的預測準確度。